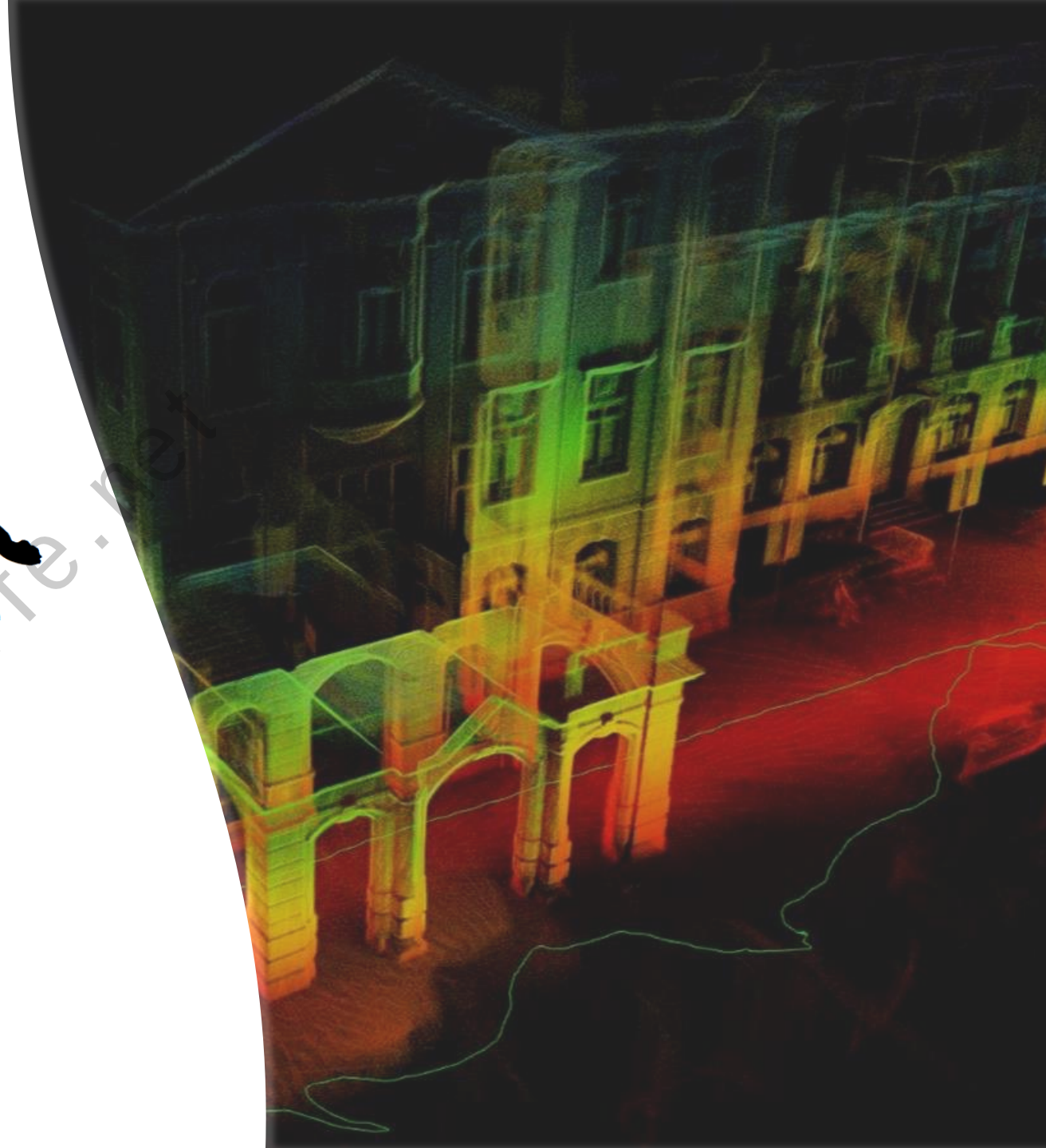


# FAST-LIO 简介

cv/ire.yat



# 什么是 FAST-LIO

- 香港大学MaRS实验室论文(IEEE-RAL,TRO)
- 高计算效率、高鲁棒性的雷达惯性里程计
- 紧耦合迭代卡尔曼滤波融合IMU和LiDAR
- FAST-LIO2在FAST-LIO基础上，提出了ikdtree数据结构
- 实现增量式的地图更新
- 代码已开源至Github

**FAST-LIO: A Fast, Robust LiDAR-inertial Odometry Package by  
Tightly-Coupled Iterated Kalman Filter**

Wei Xu<sup>1</sup>, Fu Zhang<sup>1</sup>

**FAST-LIO2: Fast Direct LiDAR-inertial Odometry**

Wei Xu<sup>\*1</sup>, Yixi Cai<sup>\*1</sup>, Dongjiao He<sup>1</sup>, Jiarong Lin<sup>1</sup>, Fu Zhang<sup>1</sup>



Scan rate: 100 Hz    Replay: X10

White Points: Current Scan (All Raw Points)  
Color Points: All the Historical Raw Points

Average Processing time:

Intel i7: 1.82 ms

ARM: 5.23 ms



# 什么是 FAST-LIO

- 香港大学MaRS实验室论文(IEEE-RAL,TRO)
- 高计算效率、高鲁棒性的雷达惯性里程计
- 紧耦合迭代卡尔曼滤波融合IMU和LiDAR
- FAST-LIO2在FAST-LIO基础上，提出了ikdtree数据结构
- 实现增量式的地图更新
- 代码已开源至Github

**FAST-LIO: A Fast, Robust LiDAR-inertial Odometry Package by  
Tightly-Coupled Iterated Kalman Filter**

Wei Xu<sup>1</sup>, Fu Zhang<sup>1</sup>

**FAST-LIO2: Fast Direct LiDAR-inertial Odometry**

Wei Xu<sup>\*1</sup>, Yixi Cai<sup>\*1</sup>, Dongjiao He<sup>1</sup>, Jiarong Lin<sup>1</sup>, Fu Zhang<sup>1</sup>



Scan rate: 100 Hz    Replay: X10

White Points: Current Scan (All Raw Points)  
Color Points: All the Historical Raw Points

Average Processing time:

Intel i7: 1.82 ms

ARM: 5.23 ms



# 为什么学习FAST-LIO

- 与其它开源框架相比，具有更优越的表现（精度、计算速度）
- 是目前最先进的开源LIO框架之一
- 涵盖流形、李群李代数、IMU积分、雷达残差、卡尔曼滤波等多方位的知识
- 学会FAST-LIO有助于理解其它多传感器融合框架
- 代码可读性较好，适合初学者上手

TABLE VII  
MEAN TIME CONSUMPTION IN MILLISECONDS BY INDIVIDUAL COMPONENTS WHEN PROCESSING A LiDAR SCAN

|                     | FAST-LIO        | FAST-LIO2      |                |
|---------------------|-----------------|----------------|----------------|
|                     | Intel           | Intel          | ARM            |
| Preprocessing       | 0.03 <i>ms</i>  | 0.03 <i>ms</i> | 0.05 <i>ms</i> |
| Feature extraction  | 0.90 <i>ms</i>  | 0 <i>ms</i>    | 0 <i>ms</i>    |
| State estimation    | 0.99 <i>ms</i>  | 1.66 <i>ms</i> | 4.75 <i>ms</i> |
| Mapping             | 13.81 <i>ms</i> | 0.13 <i>ms</i> | 0.43 <i>ms</i> |
| Total               | 15.83 <i>ms</i> | 1.82 <i>ms</i> | 5.23 <i>ms</i> |
| Num. of points used | 447             | 156            | 156            |
| Num. of threads     | 4               | 4              | 2              |

## FAST-LIO2平均计算时间

TABLE IV  
ABSOLUTE TRANSLATIONAL ERRORS (RMSE, METERS) IN SEQUENCES WITH GOOD QUALITY GROUND TRUTH

|                     | <i>utbm_8</i>  | <i>utbm_9</i> | <i>utbm_10</i> | <i>ulhk_4</i> | <i>nclt_4</i> | <i>nclt_5</i> | <i>nclt_6</i> | <i>nclt_7</i> | <i>nclt_8</i> | <i>nclt_9</i> | <i>nclt_10</i> | <i>liosam_1</i> |
|---------------------|----------------|---------------|----------------|---------------|---------------|---------------|---------------|---------------|---------------|---------------|----------------|-----------------|
| FAST-LIO2 (2000m)   | 25.3           | 51.6          | 16.89          | 2.57          | 8.63          | 6.66          | 21.01         | 6.59          | 30.59         | 5.72          | 17.14          | 4.62            |
| FAST-LIO2 (1000m)   | 27.29          | 51.6          | 16.8           | 2.57          | 8.71          | 6.68          | 20.96         | 6.58          | 30.08         | 5.56          | 16.29          | 4.58            |
| FAST-LIO2 (800m)    | 25.8           | 51.86         | 17.23          | 2.57          | 8.72          | 6.65          | 21.03         | 6.99          | 30.74         | 5.95          | 16.73          | 4.58            |
| FAST-LIO2 (600m)    | 27.75          | 52.09         | 17.3           | 2.57          | 8.58          | 6.69          | 20.96         | 6.82          | 30.24         | 5.8           | 16.81          | 4.58            |
| FAST-LIO2 (Feature) | 27.21          | 53.81         | 22.59          | 2.61          | 8.5           | 7.82          | 20.57         | 6.77          | 31.17         | 6.09          | 16.61          | 7.85            |
| LILI-OM             | 59.48          | 782.11        | 17.59          | 2.29          | 317.77        | 12.42         | 260.76        | 12.17         | 276.74        | 7.39          | 328.87         | 18.78           |
| LIO-SAM             | — <sup>1</sup> | —             | —              | 3.52          | 9461          | 7.15          | ×             | 22.26         | 44.83         | 7.43          | 1077.5         | 4.75            |
| LINS                | 48.17          | 54.35         | 60.48          | 3.11          | 65.95         | 1051          | 243.87        | 378.99        | 106.03        | 11.13         | 2995.9         | 880.92          |

<sup>1</sup> Dataset *utbm* does not produce the attitude quaternion data which is necessary for LIO-SAM, therefore LIO-SAM does not work on all the sequences in *utbm* dataset, denoted as —.

<sup>2</sup> × denotes that the system totally failed.

## FAST-LIO2误差对比

## ● 需要哪些预备知识

- 旋转矩阵、李群李代数、卡尔曼滤波（十四讲3-4，9）
- 传感器工作原理、模型（IMU、LiDAR）
- C++, ROS, Sophus
- 详细回顾基础知识，**初学者也能完全理解**

# ● 课程安排

- 基础知识回顾：旋转矩阵、李群李代数、Sophus库的使用
- LIO系统传感器模型（IMU、LiDAR）
- FAST-LIO代码真机测试
- FAST-LIO整体框架讲解
- 广义运算符与流形
- IMU连续模型与离散模型
- 点云预处理

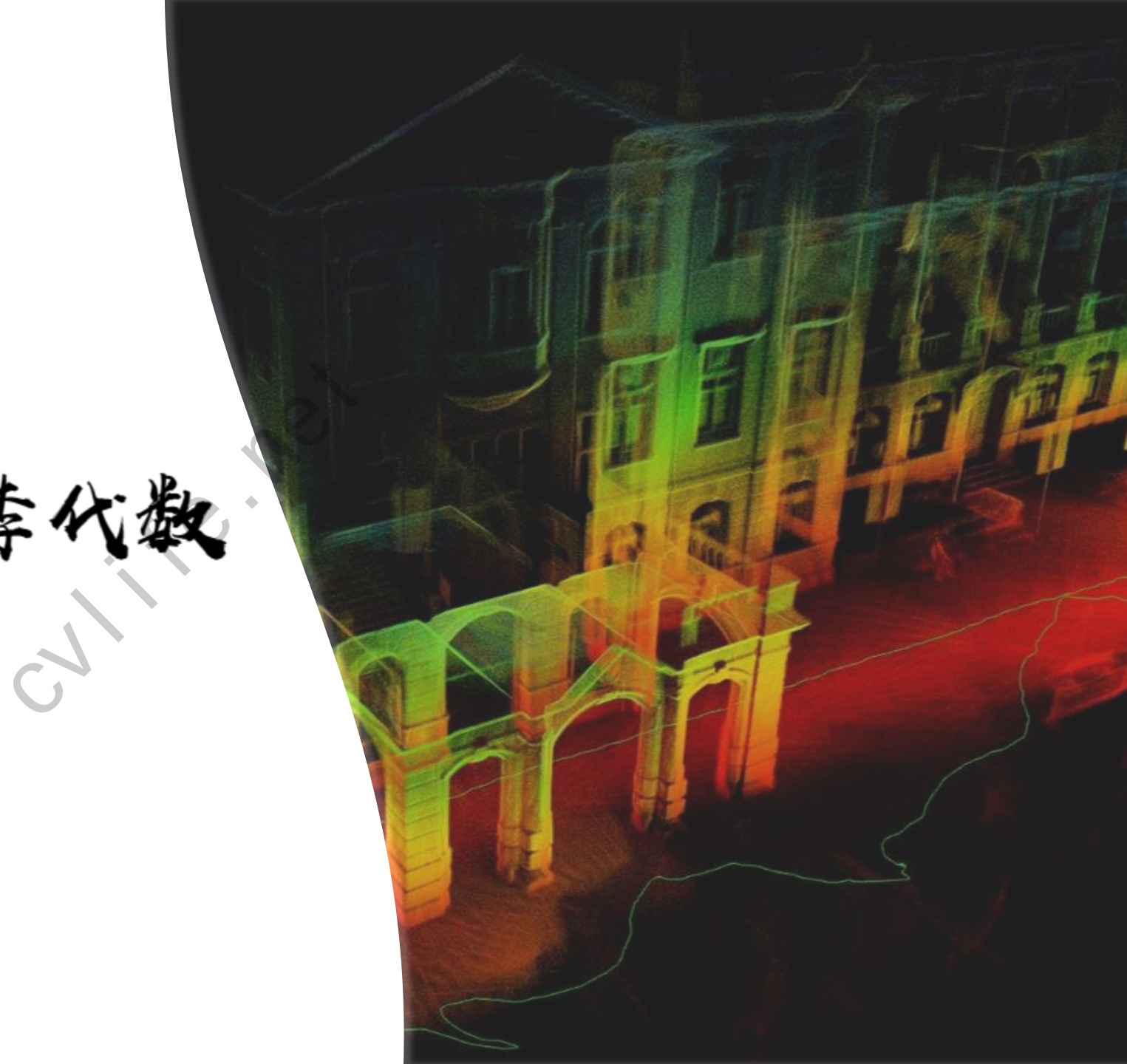
# ● 课程安排

- IMU前向传播
- IMU反向传播与运动补偿
- 卡尔曼滤波基础知识回顾
- 误差状态卡尔曼滤波器(ESKF)
- 地图更新
- ikdtree数据结构

- ✓ 每部分都包含详细的**论文解读**、**公式推导**和**代码讲解**
- ✓ 建议提前温习基础知识，和提前阅读代码
- ✓ 基础知识回顾：《视觉SLAM十四讲》第3-4章、第9章状态估计部分，IMU运动模型，LiDAR基本原理
- ✓ 预习代码：[https://github.com/zlwang7/S-FAST\\_LIO](https://github.com/zlwang7/S-FAST_LIO)
- ✓ 原版代码：[https://github.com/hku-mars/FAST\\_LIO](https://github.com/hku-mars/FAST_LIO)

谢谢

# 旋转矩阵、李群与李代数

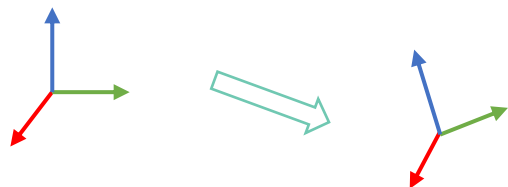


## ● 旋转矩阵

➤ 空间中向量表示方式:  $a = [e_1, e_2, e_3] \begin{bmatrix} a_1 \\ a_2 \\ a_3 \end{bmatrix} = a_1 e_1 + a_2 e_2 + a_3 e_3$

正交基

➤ 坐标系之间的欧式变换:



• 对于同一向量a

• 欧式变换: 旋转+平移

$$[e_1, e_2, e_3] \begin{bmatrix} a_1 \\ a_2 \\ a_3 \end{bmatrix} = [e'_1, e'_2, e'_3] \begin{bmatrix} a'_1 \\ a'_2 \\ a'_3 \end{bmatrix}$$

原坐标系

变换后坐标系

## ● 旋转矩阵

➤ 两边同乘  $[\mathbf{e}_1, \mathbf{e}_2, \mathbf{e}_3]^T$ :

$$\begin{bmatrix} a_1 \\ a_2 \\ a_3 \end{bmatrix} = \begin{bmatrix} \mathbf{e}_1^T \mathbf{e}'_1 & \mathbf{e}_1^T \mathbf{e}'_2 & \mathbf{e}_1^T \mathbf{e}'_3 \\ \mathbf{e}_2^T \mathbf{e}'_1 & \mathbf{e}_2^T \mathbf{e}'_2 & \mathbf{e}_2^T \mathbf{e}'_3 \\ \mathbf{e}_3^T \mathbf{e}'_1 & \mathbf{e}_3^T \mathbf{e}'_2 & \mathbf{e}_3^T \mathbf{e}'_3 \end{bmatrix} \begin{bmatrix} a'_1 \\ a'_2 \\ a'_3 \end{bmatrix} \stackrel{\text{def}}{=} \mathbf{R} \mathbf{a}'$$

旋转矩阵

➤ 旋转矩阵的性质:

$$\text{SO}(n) = \{ \mathbf{R} \in \mathbb{R}^{n \times n} \mid \mathbf{R} \mathbf{R}^T = \mathbf{I}, \det(\mathbf{R}) = 1 \}$$

特殊正交群

➤ 经过一次旋转和平移:

$$\mathbf{a}' = \mathbf{R} \mathbf{a} + \mathbf{t}$$

平移向量

## ● 旋转矩阵

➤ 如果有两次变换:  $\mathbf{R}_1, \mathbf{t}_1, \mathbf{R}_2, \mathbf{t}_2$

$$\begin{aligned}\mathbf{b} &= \mathbf{R}_1 \mathbf{a} + \mathbf{t}_1, \quad \mathbf{c} = \mathbf{R}_2 \mathbf{b} + \mathbf{t}_2 \\ \mathbf{c} &= \mathbf{R}_2 (\mathbf{R}_1 \mathbf{a} + \mathbf{t}_1) + \mathbf{t}_2\end{aligned}$$



➤ 齐次变换:

$$\begin{bmatrix} \mathbf{a}' \\ 1 \end{bmatrix} = \begin{bmatrix} \mathbf{R} & \mathbf{t} \\ \mathbf{0}^T & 1 \end{bmatrix} \begin{bmatrix} \mathbf{a} \\ 1 \end{bmatrix} \stackrel{\text{def}}{=} \mathbf{T} \begin{bmatrix} \mathbf{a} \\ 1 \end{bmatrix}$$

$$\mathbf{b} = \mathbf{T}_1 \tilde{\mathbf{a}}, \tilde{\mathbf{c}} = \mathbf{T}_2 \mathbf{b} \Rightarrow \tilde{\mathbf{c}} = \mathbf{T}_2 \mathbf{T}_1 \tilde{\mathbf{a}}$$

➤ 特殊欧式群:

$$\text{SE}(3) = \left\{ \mathbf{T} = \begin{bmatrix} \mathbf{R} & \mathbf{t} \\ \mathbf{0}^T & 1 \end{bmatrix} \in \mathbb{R}^{4 \times 4} \mid \mathbf{R} \in \text{SO}(3), \mathbf{t} \in \mathbb{R}^3 \right\}$$

# ● 李群与李代数

➤ 群的定义：一种**集合A** + 一种**运算·** 的代数结构

✓ 封闭性：  $\forall a_1, a_2 \in A, \quad a_1 \cdot a_2 \in A$

✓ 结合律：  $\forall a_1, a_2, a_3 \in A, \quad (a_1 \cdot a_2) \cdot a_3 = a_1 \cdot (a_2 \cdot a_3)$

✓ 幺元：  $\exists a_0 \in A, \quad s.t. \quad \forall a \in A, \quad a_0 \cdot a = a \cdot a_0 = a$

✓ 逆：  $\forall a \in A, \quad \exists a^{-1} \in A, \quad s.t. \quad a \cdot a^{-1} = a_0$

➤ 李群：具有**连续、光滑性质**的群

× 整数群

✓ 特殊正交群SO(3)

✓ 特殊欧式群SE(3)

□ 自行查阅：还有哪些群是李群？

# ● 李群与李代数

➤ 李代数的定义：一个集合  $\mathbb{V}$  + 一个数域  $\mathbb{F}$  + 一个二元运算  $[\cdot, \cdot]$  (李括号)

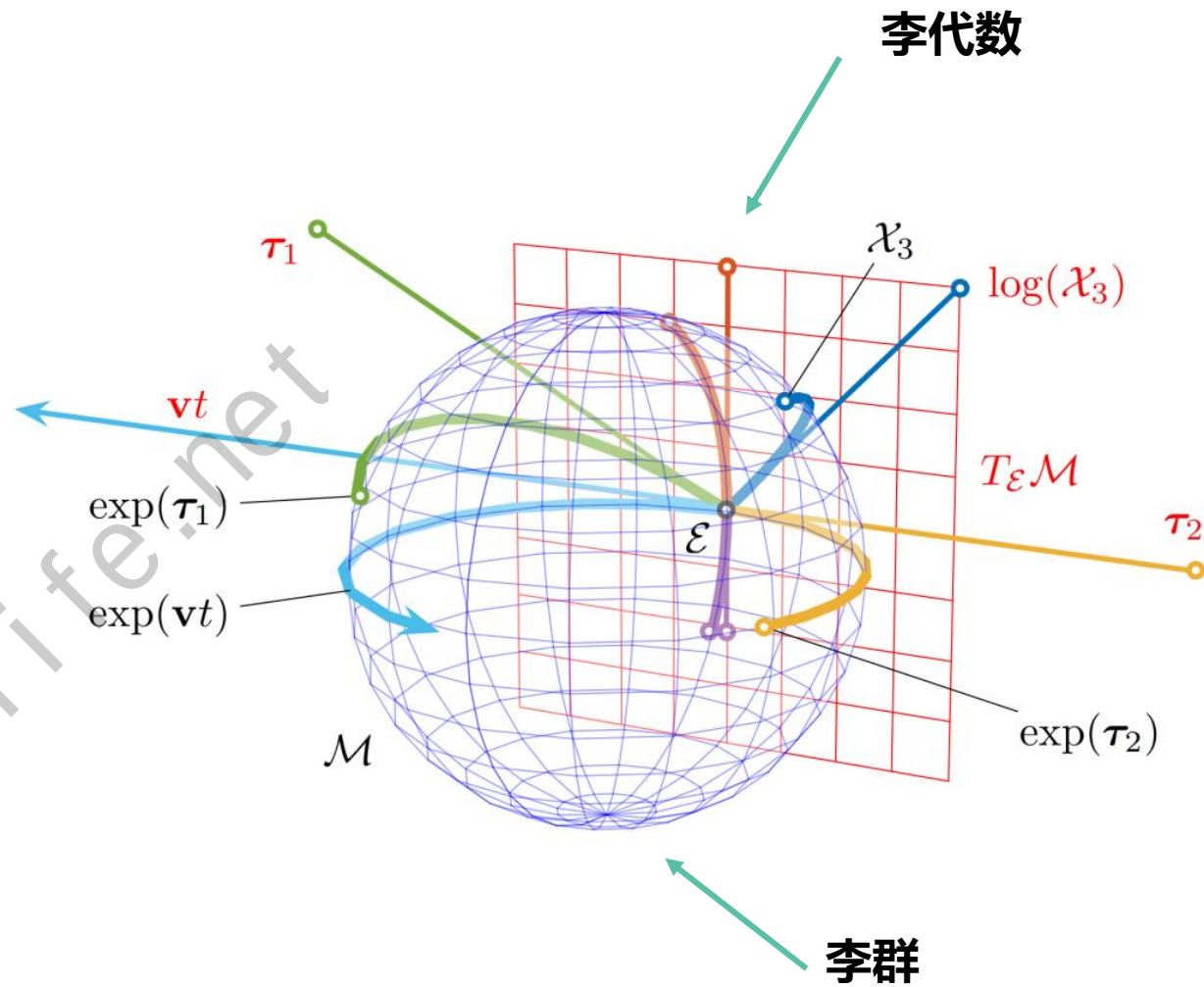
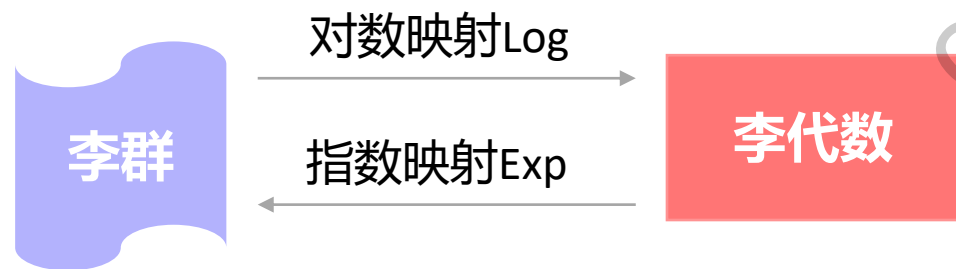
- ✓ 封闭性：  $\forall \mathbf{X}, \mathbf{Y} \in \mathbb{V}, [\mathbf{X}, \mathbf{Y}] \in \mathbb{V}$
- ✓ 双线性：  $\forall \mathbf{X}, \mathbf{Y}, \mathbf{Z} \in \mathbb{V}, a, b \in \mathbb{F} \quad [a\mathbf{X} + b\mathbf{Y}, \mathbf{Z}] = a[\mathbf{X}, \mathbf{Z}] + b[\mathbf{Y}, \mathbf{Z}], [\mathbf{Z}, a\mathbf{X} + b\mathbf{Y}] = a[\mathbf{Z}, \mathbf{X}] + b[\mathbf{Z}, \mathbf{Y}]$
- ✓ 自反性：  $\forall \mathbf{X} \in \mathbb{V}, [\mathbf{X}, \mathbf{X}] = \mathbf{0}$
- ✓ 雅可比等价：  $\forall \mathbf{X}, \mathbf{Y}, \mathbf{Z} \in \mathbb{V}, [\mathbf{X}, [\mathbf{Y}, \mathbf{Z}]] + [\mathbf{Z}, [\mathbf{X}, \mathbf{Y}]] + [\mathbf{Y}, [\mathbf{Z}, \mathbf{X}]] = \mathbf{0}$

- 每个李群都有与之对应的李代数
- 例如  $\mathbb{R}^3$  上的叉积就是一种李括号

# 李群与李代数

## 李群与李代数的关系

- 每个李群都有与之对应的李代数
- 李代数是李群在单位元处的切空间
- 李代数是线性向量空间



# 李群与李代数

## ➤ 特殊正交群SO(3)

- 李群为 $3 \times 3$ 的旋转矩阵
- 李代数是 $\mathbb{R}^3$ 上的向量

## ➤ SO(3)上的指数映射

$$R = \exp(\phi^\wedge) = \cos \theta \mathbf{I} + (1 - \cos \theta) \mathbf{a} \mathbf{a}^\top + \sin \theta \mathbf{a}^\wedge, \phi = \theta \mathbf{a}$$

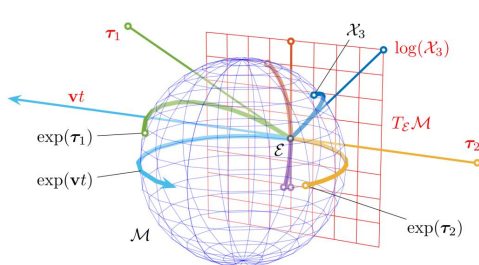
即罗德里格斯公式

## ➤ SO(3)上的对数映射

$$\theta = \arccos \frac{\text{tr}(\mathbf{R}) - 1}{2} \quad \mathbf{R} \mathbf{a} = \mathbf{a}$$

迹的性质

转轴旋转后不变



对数映射Log

指数映射Exp

李代数

$$\phi = \theta \mathbf{a} \in \mathbb{R}^3$$

反对称阵

$$\mathbf{a}^\wedge = \begin{bmatrix} 0 & -a_3 & a_2 \\ a_3 & 0 & -a_1 \\ -a_2 & a_1 & 0 \end{bmatrix} \in \mathbb{R}^{3 \times 3}.$$

转角

单位方向向量 (转轴)

# ● 李群与李代数

## ➤ BCH近似公式

×  $SO(3)$ 上的矩阵乘法  $\neq$  李代数上的加法:  $\exp(\phi_1^\wedge)\exp(\phi_2^\wedge) \neq \exp(\phi_1 + \phi_2)^\wedge$

$$\ln\left(\exp(\phi_1^\wedge)\exp(\phi_2^\wedge)\right)^\vee \approx \begin{cases} J_l(\phi_2)^{-1}\phi_1 + \phi_2 & \phi_1 \text{ 为小量} \\ \phi_1 + J_r(\phi_1)^{-1}\phi_2 & \phi_2 \text{ 为小量} \end{cases}$$

左乘雅可比      右乘雅可比

## ➤ 近似雅可比

$$\mathbf{J}_l^{-1} = \frac{\theta}{2} \cot \frac{\theta}{2} \mathbf{I} + \left(1 - \frac{\theta}{2} \cot \frac{\theta}{2}\right) \mathbf{a} \mathbf{a}^\top - \frac{\theta}{2} \mathbf{a}^\wedge.$$

$$J_r(\phi) = J_l(-\phi).$$

# ● 李群与李代数

➤ SO(3) 求导: Rp对R求导

a. 用李代数表示姿态，根据李代数加法求导

设矩阵R对应的李代数为  $\phi$

$$\begin{aligned}
 \frac{\partial(\exp(\phi^\wedge)\mathbf{p})}{\partial\phi} &= \lim_{\delta\phi \rightarrow 0} \frac{\exp((\phi + \delta\phi)^\wedge)\mathbf{p} - \exp(\phi^\wedge)\mathbf{p}}{\delta\phi} \\
 &= \lim_{\delta\phi \rightarrow 0} \frac{\exp((\mathbf{J}_l\delta\phi)^\wedge)\exp(\phi^\wedge)\mathbf{p} - \exp(\phi^\wedge)\mathbf{p}}{\delta\phi} \\
 &= \lim_{\delta\phi \rightarrow 0} \frac{(\mathbf{I} + (\mathbf{J}_l\delta\phi)^\wedge)\exp(\phi^\wedge)\mathbf{p} - \exp(\phi^\wedge)\mathbf{p}}{\delta\phi} \\
 &= \lim_{\delta\phi \rightarrow 0} \frac{(\mathbf{J}_l\delta\phi)^\wedge \exp(\phi^\wedge)\mathbf{p}}{\delta\phi} \\
 &= \lim_{\delta\phi \rightarrow 0} \frac{-(\exp(\phi^\wedge)\mathbf{p})^\wedge \mathbf{J}_l\delta\phi}{\delta\phi} \\
 &= -(\mathbf{Rp})^\wedge \mathbf{J}_l
 \end{aligned}$$

BCH近似

Taylor展开

反对称性质

b. 对李群**左乘或右乘微小扰动**，对扰动求导

设矩阵R对应的李代数为  $\phi$ ，扰动的李代数为  $\varphi$

$$\begin{aligned}
 \frac{\partial(\mathbf{Rp})}{\partial\varphi} &= \lim_{\varphi \rightarrow 0} \frac{\exp(\varphi^\wedge)\exp(\phi^\wedge)\mathbf{p} - \exp(\phi^\wedge)\mathbf{p}}{\varphi} \\
 &= \lim_{\varphi \rightarrow 0} \frac{(\mathbf{I} + \varphi^\wedge)\exp(\phi^\wedge)\mathbf{p} - \exp(\phi^\wedge)\mathbf{p}}{\varphi} \\
 &= \lim_{\varphi \rightarrow 0} \frac{\varphi^\wedge \mathbf{Rp}}{\varphi} \\
 &= \lim_{\varphi \rightarrow 0} \frac{-(\mathbf{Rp})^\wedge \varphi}{\varphi} \\
 &= -(\mathbf{Rp})^\wedge
 \end{aligned}$$

Taylor展开

反对称性质

# ● Sophus库

- 在Eigen基础上，提供李群、李代数运算

Sophus安装：

```
git clone https://github.com/strasdat/Sophus.git
cd Sophus
git checkout a621ff
mkdir build
cd build
cmake ../ -DUSE_BASIC_LOGGING=ON
make
sudo make install
```

使用示例：

```
#include <iostream>
using namespace std;
#include <Eigen/Core>
#include "sophus/so3.h"
#include "sophus/se3.h"
int main( int argc, char** argv )
{
    // 沿Y轴转45度的旋转矩阵
    Eigen::Matrix3d R = Eigen::AngleAxisd(M_PI/4,
    Eigen::Vector3d(0,1,0)).toRotationMatrix();
    //Sophus的构造方式
    Sophus::SO3 rot_r(R);           // 从旋转矩阵构造
    Sophus::SO3 SO3_v(0, M_PI/4, 0); // 从旋转向量构造
    Eigen::Quaterniond q(R);        // 从四元数构造
    Sophus::SO3 SO3_q( q );

    // 输出时，以李代数输出
    cout << "SO(3) from matrix: " << rot_r << endl;
    cout << "SO(3) from vector: " << SO3_v << endl;
    cout << "SO(3) from quaternion : " << SO3_q << endl;

    // 也可以输出矩阵
    cout << "SO(3) matrix : "<<rot_r.matrix() << endl << endl;
    // 对数映射 获取李代数
    Eigen::Vector3d fai = rot_r.log();
    cout<<"log(rot_r) = " <<fai.transpose()<<endl<<endl;
    // 求hat (反对称矩阵)
    cout<<"fai hat= " <<Sophus::SO3::hat(fai)<<endl<<endl;
    // 从反对称到向量
    cout<<"fai hat vee=
    "<<Sophus::SO3::vee( Sophus::SO3::hat(fai) ).transpose()<<endl<<endl;
    // 指数映射 从李代数到SO3
    Sophus::SO3 rot_R = Sophus::SO3::exp(fai);
    cout<<"exp(fai) = " <<rot_R<<endl;

    //直接相乘
    Sophus::SO3 rot_R1(M_PI/3, M_PI/3, 0);
    Sophus::SO3 RR = rot_R * rot_R1;
    cout<<"RR = " <<RR<<endl;
    cout<<"RR.matrix = " <<RR.matrix()<<endl<<endl;
    cout<<"rot_R.matrix * rot_R1.matrix = " <<rot_R.matrix() * rot_R1.matrix() <<endl;
    return 0;
}
```

谢谢

LIO 系统传感器模型



# ● IMU (惯性测量单元)

➤ 测量物体三轴姿态角(或角速率)以及加速度的装置

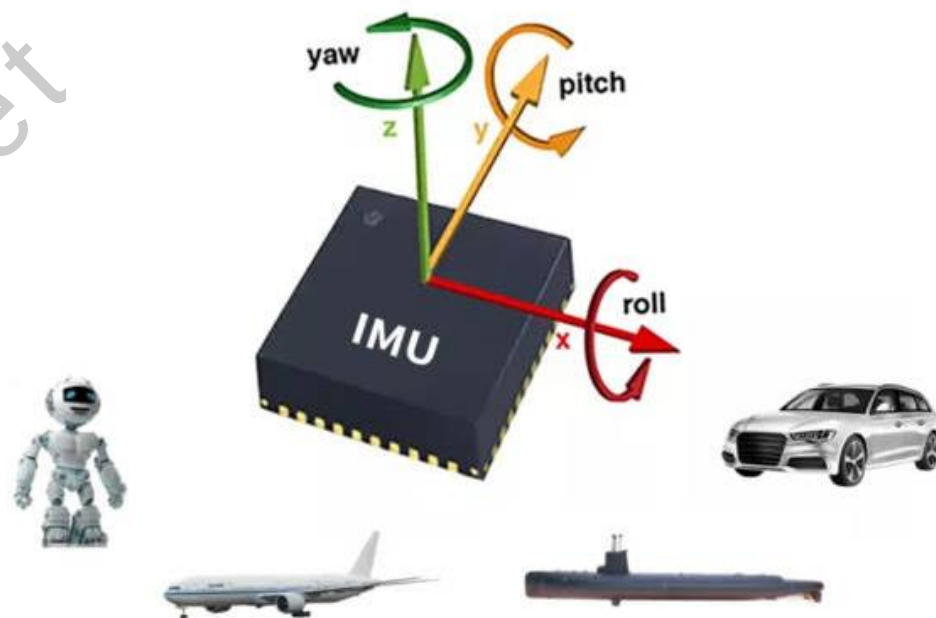
□ 6轴： 三轴加速度计 + 三轴陀螺仪(角速度传感器)

□ 9轴： 6轴 + 三轴磁力计 (角度)

➤ 为什么用IMU

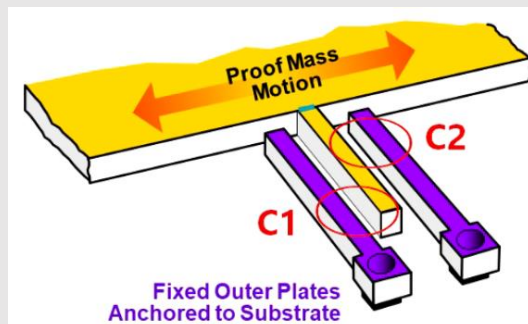
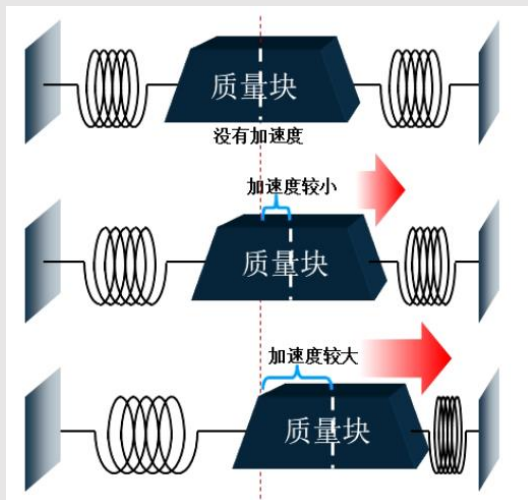
- ✓ 输出加速度角速度
- ✓ 频率高 (100-400HZ)
- ✓ 不受外界干扰
- ✓ 价格不贵

cvlife.net



# ● IMU测量原理

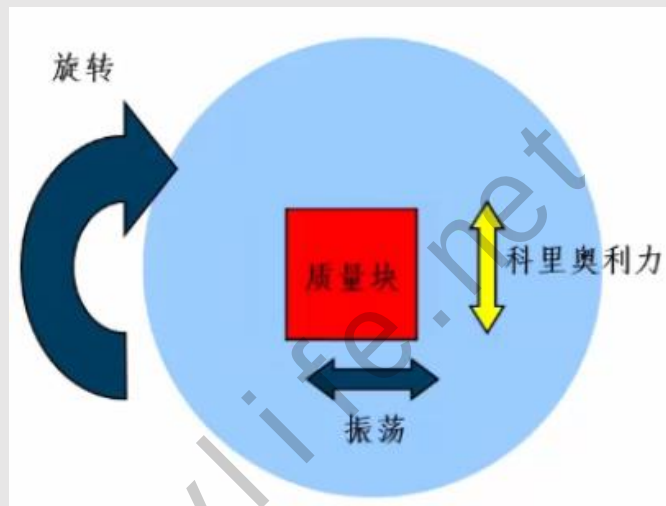
## ➤ MEMS加速度计工作原理



□  $F = ma = kx$

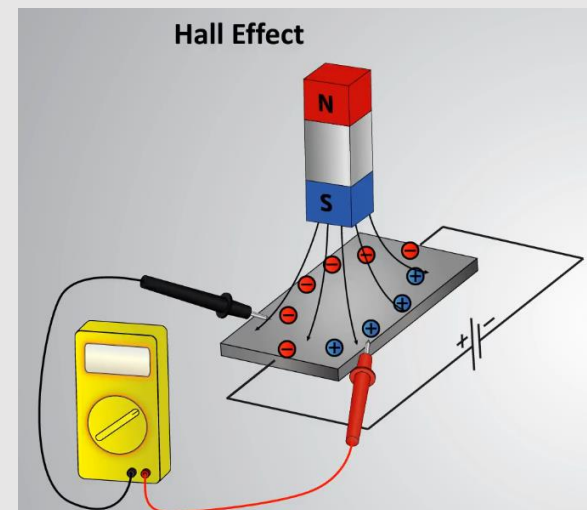
□ 通过电容差计算位移

## ➤ 陀螺仪工作原理



□ 物体周期运动时，在其正交平面内旋转物体，会在与物体周期运动的垂直方向上产生科氏力

## ➤ 磁力计工作原理



□ 霍尔效应测磁场强度  
□ 进而得到所需的载体方位角

# ● IMU误差模型

## ➤ 确定性误差

- Bias
- Scale

## ➤ 随机误差

- 高斯白噪声
- Bias随机游走

### Bias

理论上静止时，IMU的输出不为0，存在偏置 $b$

### Scale

实际数值和传感器输出值之间的比值

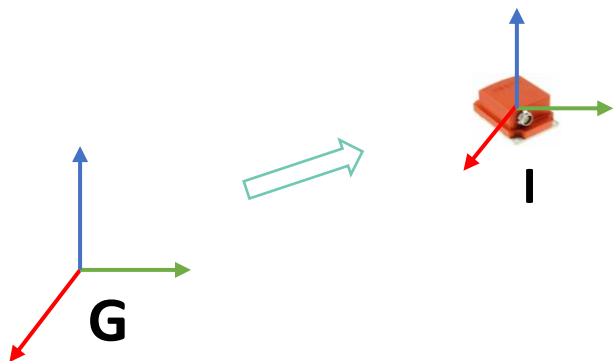
### 高斯白噪声

IMU 数据连续时间上受到均值为0的高斯噪声

### Bias随机游走

Bias随时间连续变化，bias的导数为高斯分布

# ● IMU测量模型



➤ 测量值 = 真值 + 零偏 + 噪声

$$\mathbf{a}_m = \mathbf{a} + \mathbf{b}_a + \mathbf{n}_a - {}^I \mathbf{R}_G {}^G \mathbf{g}$$

$$\boldsymbol{\omega}_m = \boldsymbol{\omega} + \mathbf{b}_w + \mathbf{n}_w$$

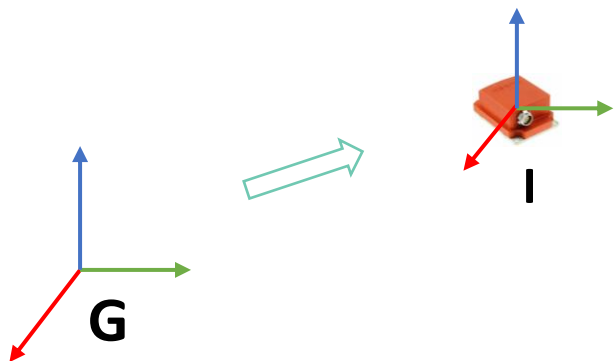
$$\dot{\mathbf{b}}_w = \mathbf{n}_{bw}, \dot{\mathbf{b}}_a = \mathbf{n}_{ba}$$

重力向量

Bias导数服从高斯分布

➤ 通常忽略Scale，只考虑bias和高斯噪声

# IMU连续运动模型

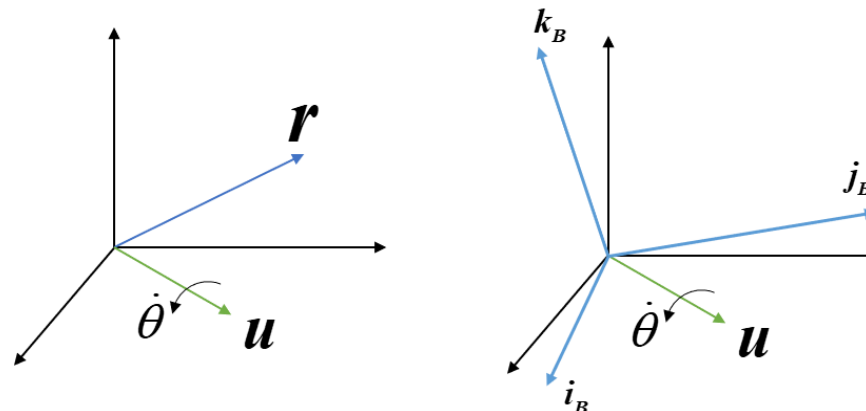


➤ 在G系下,连续运动的导数:

$$\dot{\mathbf{R}} = \mathbf{R}\boldsymbol{\omega}^\wedge$$

$$\dot{\mathbf{v}} = \mathbf{a}$$

$$\dot{\mathbf{p}} = \mathbf{v}$$



考虑一个从原点出发的向量 $\mathbf{r}$ 绕单位轴 $\mathbf{u}$ 旋转, 角速度为  $\boldsymbol{\omega} = \dot{\theta}\mathbf{u}$ , 角速度大小为  $\dot{\theta}$ , 则:

$$\frac{d\mathbf{r}}{dt} = \boldsymbol{\omega} \times \mathbf{r}$$

坐标系绕单位轴 $\mathbf{u}$ 旋转, 其三个轴的导数:

$$\frac{d\mathbf{i}_B}{dt} = \boldsymbol{\omega} \times \mathbf{i}_B, \frac{d\mathbf{j}_B}{dt} = \boldsymbol{\omega} \times \mathbf{j}_B, \frac{d\mathbf{k}_B}{dt} = \boldsymbol{\omega} \times \mathbf{k}_B$$

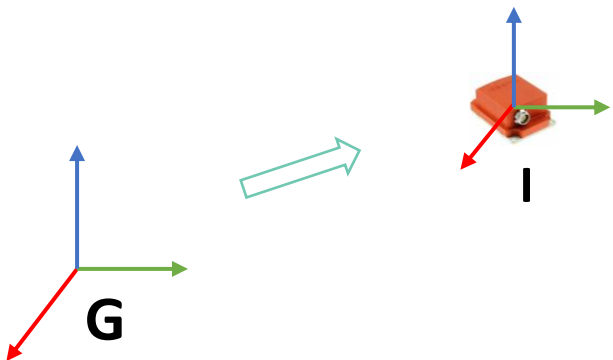
$[\mathbf{i}_B, \mathbf{j}_B, \mathbf{k}_B]$  实际上就是坐标系  $\{B\}$  相对于原坐标系的旋转矩阵  $\mathbf{R}$

$$\dot{\mathbf{R}} = \begin{bmatrix} \boldsymbol{\omega} \times \mathbf{i}_B & \boldsymbol{\omega} \times \mathbf{j}_B & \boldsymbol{\omega} \times \mathbf{k}_B \end{bmatrix} = \boldsymbol{\omega} \times \mathbf{R} = \boldsymbol{\omega}^\wedge \mathbf{R}$$

这里的 $\boldsymbol{\omega}$ 是原坐标系下的, 根据  $(R\mathbf{p})^\wedge = R\mathbf{p}^\wedge R^T$

$${}^G\dot{\mathbf{R}}_I = ({}^G\mathbf{R}_I \cdot {}^I\boldsymbol{\omega})^\wedge {}^G\mathbf{R}_I = {}^G\mathbf{R}_I ({}^I\boldsymbol{\omega})^\wedge$$

## ● IMU离散模型



➤ 离散化：

$$\mathbf{R}(t + \Delta t) = \mathbf{R} \text{Exp} \left( \int_t^{t+\Delta t} \boldsymbol{\omega}(\tau) d\tau \right)$$

$$\mathbf{v}(t + \Delta t) = \mathbf{v}(t) + \int_t^{t+\Delta t} \mathbf{a}(\tau) d\tau$$

$$\mathbf{p}(t + \Delta t) = \mathbf{p}(t) + \int_t^{t+\Delta t} \mathbf{v}(\tau) d\tau + \iint_t^{t+\Delta t} \mathbf{a}(\tau) d\tau^2$$

欧拉积分（两个相邻时刻k到k+1的位姿用k时刻的测量值计算）：

$$\dot{\mathbf{R}} = \mathbf{R} \boldsymbol{\omega}^\wedge$$

$$\dot{\mathbf{v}} = \mathbf{a}$$

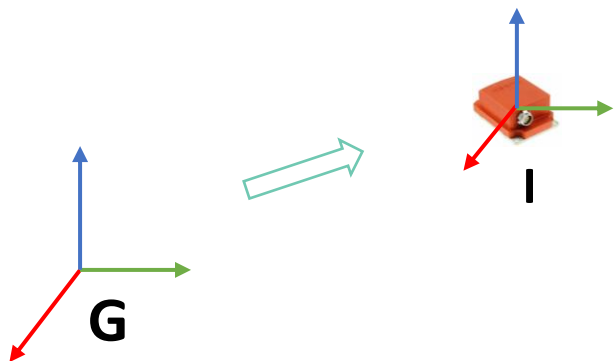
$$\dot{\mathbf{p}} = \mathbf{v}$$

$$\mathbf{R}(t + \Delta t) = \mathbf{R}(t) \text{Exp}(\boldsymbol{\omega}(t) \Delta t)$$

$$\mathbf{v}(t + \Delta t) = \mathbf{v}(t) + \mathbf{a}(t) \Delta t$$

$$\mathbf{p}(t + \Delta t) = \mathbf{p}(t) + \mathbf{v}(t) \Delta t + \frac{1}{2} \mathbf{a}(t) \Delta t^2$$

# ● IMU离散模型



$$\dot{\mathbf{R}} = \mathbf{R}\boldsymbol{\omega}^{\wedge}$$

$$\dot{\mathbf{v}} = \mathbf{a}$$

$$\dot{\mathbf{p}} = \mathbf{v}$$

➤ 测量模型:

$$\mathbf{a}_m = \mathbf{a} + \mathbf{b}_a + \mathbf{n}_a - {}^I \mathbf{R}_G {}^G \mathbf{g}$$

$$\boldsymbol{\omega}_m = \boldsymbol{\omega} + \mathbf{b}_w + \mathbf{n}_w$$

$$\dot{\mathbf{b}}_w = \mathbf{n}_{bw}, \dot{\mathbf{b}}_a = \mathbf{n}_{ba}$$

➤ 离散模型:

$$\mathbf{R}(t + \Delta t) = \mathbf{R}(t) \text{Exp}(\boldsymbol{\omega}(t)\Delta t)$$

$$\mathbf{v}(t + \Delta t) = \mathbf{v}(t) + \mathbf{a}(t)\Delta t$$

$$\mathbf{p}(t + \Delta t) = \mathbf{p}(t) + \mathbf{v}(t)\Delta t + \frac{1}{2}\mathbf{a}(t)\Delta t^2$$

这里的a是G系下的a

➤ 将测量模型代入:

$$\mathbf{R}(t + \Delta t) = \mathbf{R}(t) \text{Exp}[(\boldsymbol{\omega}_m - \mathbf{b}_w - \mathbf{n}_w)\Delta t]$$

$$\mathbf{v}(t + \Delta t) = \mathbf{v}(t) + [\mathbf{R}(t)(\mathbf{a}_m - \mathbf{b}_a - \mathbf{n}_a) + {}^G \mathbf{g}]\Delta t$$

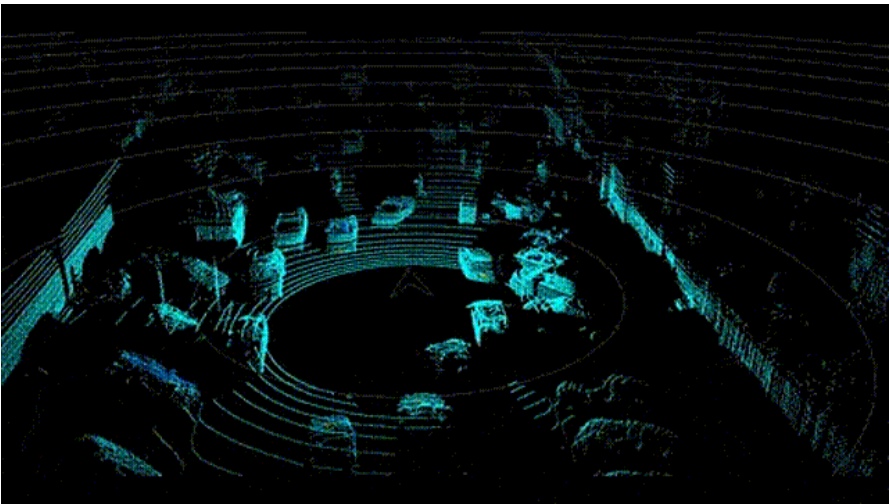
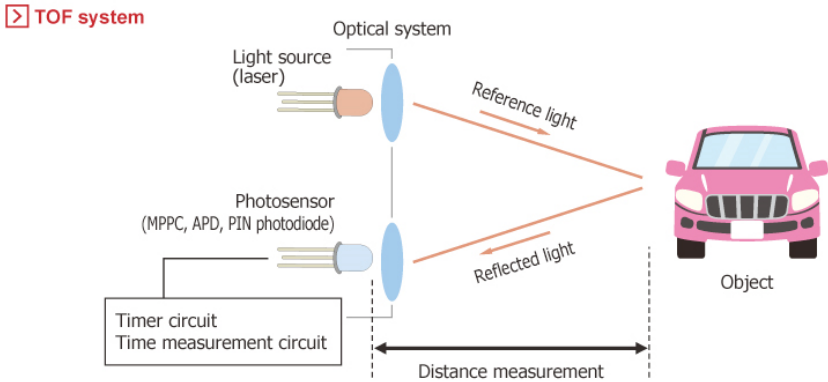
$$\mathbf{p}(t + \Delta t) = \mathbf{p}(t) + \mathbf{v}(t)\Delta t$$

IMU积分: 每收到一帧IMU数据执行一次上式

# ● 激光雷达 (LiDAR)

- Lidar: “Light Detection and Ranging”
- “光学雷达”，通过照射脉冲激光，测量发送和接受到的脉冲信号的时间间隔来计算距离
- 输出点云数据

| 型号       | RS-Helios 5515      | RS-Helios 1615      |
|----------|---------------------|---------------------|
| 线数       | 32                  | 32                  |
| 激光波长     | 905nm               | 905nm               |
| 激光安全等级   | Class 1 人眼安全        | Class 1 人眼安全        |
| 测距能力     | 150m(110m@10% NIST) | 150m(110m@10% NIST) |
| 盲区       | ≤0.2m               | ≤0.2m               |
| 精度 (典型值) | ±3cm( 3σ )          | ±3cm( 3σ )          |
| 水平视场角    | 360°                | 360°                |
| 垂直视场角    | 70° (-55°~+ 15°)    | 31° (-16°~+ 15°)    |
| 水平角分辨率   | 0.2°/0.4°           | 0.2°/0.4°           |
| 垂直角分辨率   | Up to 1.33°         | 均匀 1°               |
| 帧率       | 10Hz/20 Hz          | 10Hz/20 Hz          |

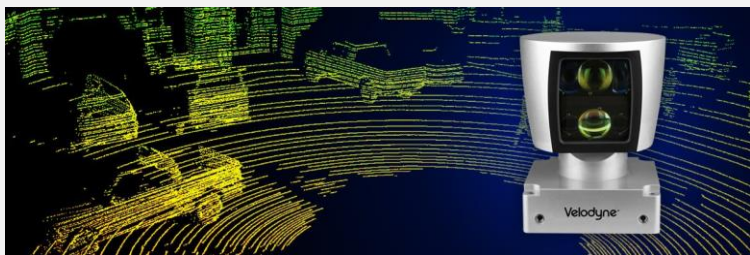


# ● 激光雷达工作原理

## 机械式



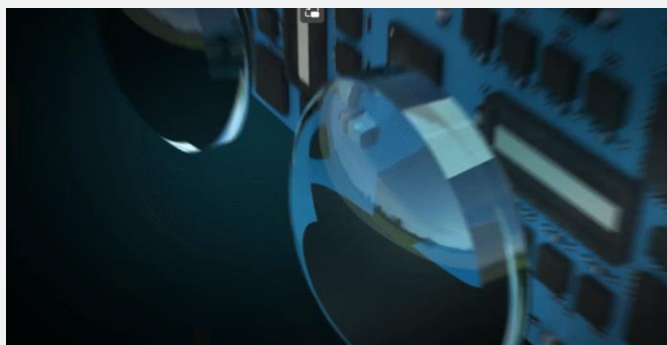
- ❑ 通过旋转激光发射和接收器，测量激光发出和收到的时间差
- ❑ 根据线数需多组激光收发模块
- ❑ 单组激光模块就有较高的成本



## 固态式



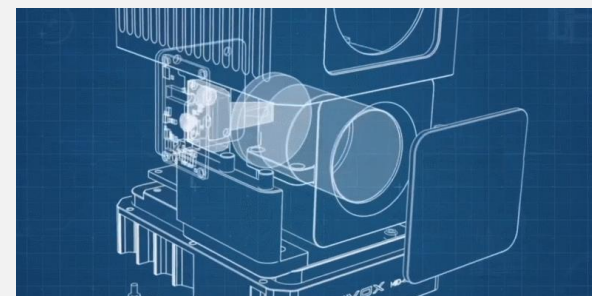
- ❑ 类似拍照，发出发散的激光
- ❑ 成本大幅降低，符合车规级
- ❑ 受限于功率，探测距离短，精度不高；光学元件制造难度大



## 混合固态



- ❑ 通过两个楔形棱镜对激光进行折射，调整棱镜转速以控制扫描区域
- ❑ 旋转部件体积小，成本低
- ❑ 寿命更长，符合车规



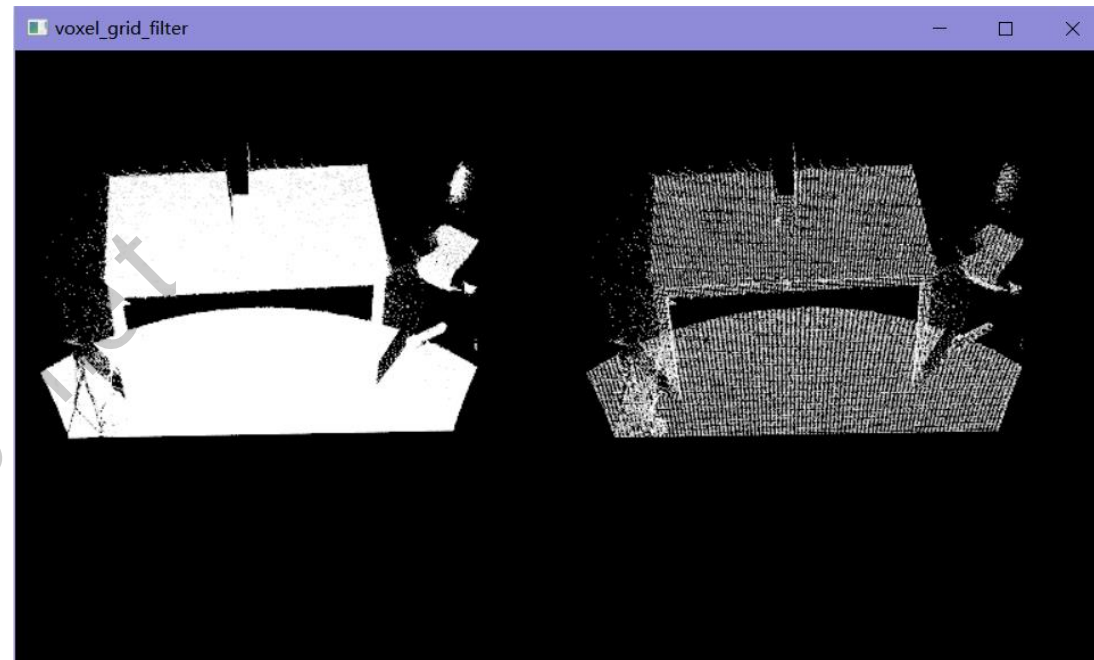
# ● 点云下采样

## ➤ 体素滤波

- ❑ 减少点的数量，增加处理速度
- ❑ 创建三维体素栅格，用体素重心近似体素内其它点
- ❑ 体素越大，下采样效果越明显

## ➤ 直通滤波

- ❑ 去掉指定范围内（或外）部的点
- ❑ 去除近处的点、地面点、离群点

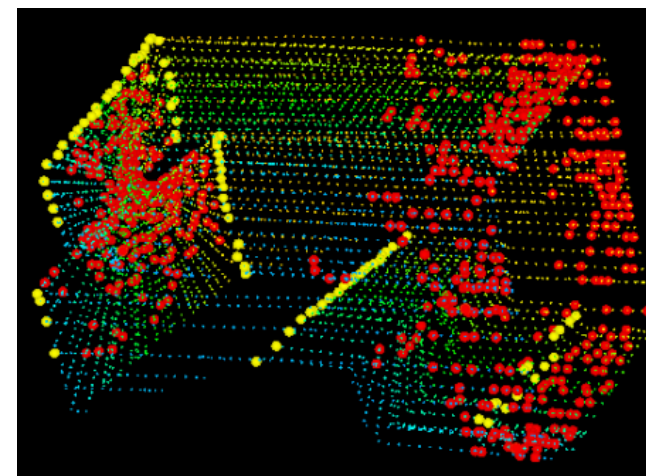
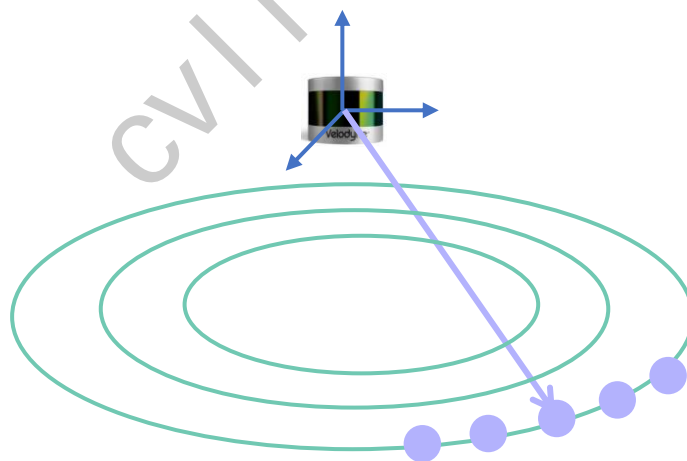


# ● 特征点提取

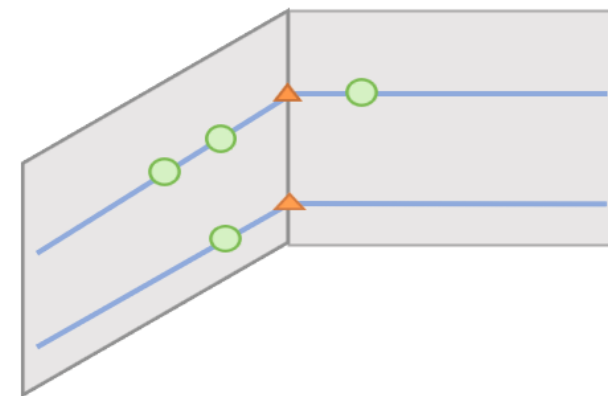
- 角点(edge point): 曲率变化较大
- 面点(planar point): 曲率变化较小

$$c = \frac{1}{|S| \cdot \|X_{(k,i)}^L\|} \left\| \sum_{j \in S, j \neq i} (X_{(k,i)}^L - X_{(k,j)}^L) \right\|.$$

- S: 相同scan的近邻点集
- 曲率判据: 点坐标变化的剧烈程度
- 小于阈值为面点, 反之为角点



- ▲ Corner point
- Plane point
- Laser beam



## ● 残差的计算

- 角点：点到近邻直线的距离
- 面点：点到近邻平面的距离
- 通过优化位姿，使残差接近零

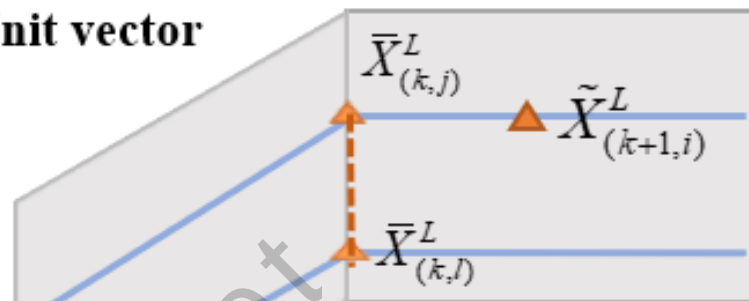
$$d_{\varepsilon} = \frac{|(\tilde{\mathbf{X}}_{(k+1,i)}^L - \bar{\mathbf{X}}_{(k,j)}^L) \times (\tilde{\mathbf{X}}_{(k+1,i)}^L - \bar{\mathbf{X}}_{(k,l)}^L)|}{|\bar{\mathbf{X}}_{(k,j)}^L - \bar{\mathbf{X}}_{(k,l)}^L|}$$

乘了待优化位姿的坐标点

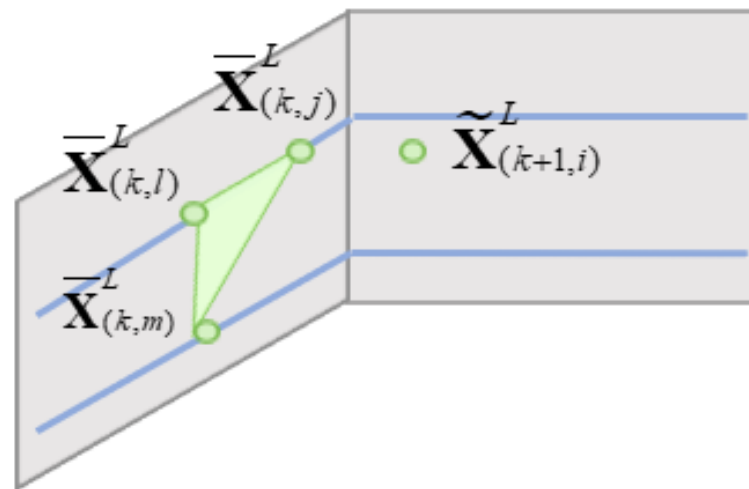
点乘

$$d_{\mathcal{H}} = \frac{\left| \left( \mathbf{X}_{(k+1,i)}^L - \bar{\mathbf{X}}_{(k,j)}^L \right) \times \left( \bar{\mathbf{X}}_{(k,j)}^L - \bar{\mathbf{X}}_{(k,l)}^L \right) \right|}{\left| \left( \bar{\mathbf{X}}_{(k,j)}^L - \bar{\mathbf{X}}_{(k,l)}^L \right) \times \left( \bar{\mathbf{X}}_{(k,j)}^L - \bar{\mathbf{X}}_{(k,m)}^L \right) \right|}$$

- ▲ Corner point
- Plane point
- Laser beam
- Unit vector



平行四边形  
面积/底边



四面体体积  
/底面面积

# 代码实战：ROS获取IMU、LiDAR数据并预处理

```
#include <fstream>
#include <iostream>
#include <ros/ros.h>
#include <Eigen/Core>
#include <nav_msgs/Odometry.h>
#include <nav_msgs/Path.h>
#include <visualization_msgs/Marker.h>
#include <pcl_conversions/pcl_conversions.h>
#include <pcl/point_cloud.h>
#include <pcl/point_types.h>
#include <pcl/filters/voxel_grid.h>
#include <pcl/io/pcd_io.h>
#include <sensor_msgs/Imu.h>
#include <sensor_msgs/PointCloud2.h>
```

```
ros::Publisher pubLaserCloud;
```

```
namespace rslidar_ros
{
    struct EIGEN_ALIGN16 Point
    {
        PCL_ADD_POINT4D;
        std::uint8_t intensity;
        std::uint16_t ring = 0;
        double timestamp = 0;
        EIGEN_MAKE_ALIGNED_OPERATOR_NEW
    };
} // namespace rslidar_ros
POINT_CLOUD_REGISTER_POINT_STRUCT(rslidar_ros::Point,
    (float, x, x)(float, y, y)(float, z, z)(std::uint8_t, intensity, intensity)(std::uint16_t, ring, ring)(double,
timestamp, timestamp))
```

```
bool first_imu = true;
double first_imu_time;
void ImuCallback(const sensor_msgs::Imu &msg)
{
    if (first_imu)
    {
        first_imu_time = msg.header.stamp.toSec();
        first_imu = false;
    }
    std::cout << "IMU time: " << msg.header.stamp.toSec() - first_imu_time << std::endl;
    std::cout << "angular_velocity: " << msg.angular_velocity.x << " " << msg.angular_velocity.y << " " <<
msg.angular_velocity.z << std::endl;
    std::cout << "linear_acceleration: " << msg.linear_acceleration.x << " " << msg.linear_acceleration.y << " " <<
msg.linear_acceleration.z << std::endl;
}
```

```
bool first_lidar = true;
double first_lidar_time;
pcl::VoxelGrid<pcl::PointXYZ> downSizeFilter;
pcl::PointCloud<pcl::PointXYZ> pl_filtered;
void laser_callback(const sensor_msgs::PointCloud2 &msg)
{
    if (first_lidar)
    {
        first_lidar_time = msg.header.stamp.toSec();
        first_lidar = false;
    }
    std::cout << "LiDAR time: " << msg.header.stamp.toSec() - first_lidar_time << std::endl;
    pcl::PointCloud<rslidar_ros::Point> pl_orig;
    pcl::fromROSMsg(msg, pl_orig);
    int plsize = pl_orig.points.size();
    std::cout << "LiDAR point size: " << plsize << std::endl;
    pl_filtered.clear();
    for (int i = 0; i < plsize; i++)
    {
        pcl::PointXYZ point;
        point.x = pl_orig.points[i].x;
        point.y = pl_orig.points[i].y;
        point.z = pl_orig.points[i].z;
        point.intensity = pl_orig.points[i].intensity;

        pl_filtered.points.push_back(point);
    }

    downSizeFilter.setLeafSize(0.05, 0.05, 0.05);
    downSizeFilter.setInputCloud(pl_filtered.makeShared());
    downSizeFilter.filter(pl_filtered);

    std::cout << "LiDAR filtered size: " << pl_filtered.points.size() << std::endl;

    sensor_msgs::PointCloud2 laserCloud;
    pcl::toROSMsg(pl_filtered, laserCloud);
    laserCloud.header.stamp = msg.header.stamp;
    laserCloud.header.frame_id = "camera_init";
    pubLaserCloud.publish(laserCloud);
}

int main(int argc, char **argv)
{
    ros::init(argc, argv, "lio_test1");
    ros::NodeHandle n("~");
    ros::Subscriber sub0 = n.subscribe("/imu/data", 10, ImuCallback);
    ros::Subscriber sub1 = n.subscribe("/rslidar_points", 10, laser_callback);
    pubLaserCloud = n.advertise<sensor_msgs::PointCloud2>("/cloud", 100000);

    while (ros::ok())
    {
        ros::spinOnce();
    }

    return 0;
}
```

谢谢

FAST-LIO代码真机测试

CODE

## ● S-FAST-LIO与原版代码的区别

**S-FAST\_LIO** is a simplified implementation of FAST\_LIO (Xu, Wei, et al. "Fast-lio2: Fast direct lidar-inertial odometry."), which is modified from FAST\_LIO. This code is clean and accessible.

It is a reference material for SLAM beginners. The main modifications are as follows:

- ✓ The Sophus is used to define the state variables, instead of the complicated IKFOM
- ✓ The gravity component is directly defined by a Vector3d, thus the complicated calculation of two-dimensional manifold can be omitted
- ✓ The code structure has been optimized, and the unnecessary codes have been deleted
- ✓ Detailed Chinese notes are added to the code
- ✓ Support for Robosense LiDAR has been added

- ✓ 新版代码: [zhwang7/S-FAST\\_LIO: A simplified implementation of FAST\\_LIO \(with Chinese note\) \(github.com\)](https://github.com/zhw7/S-FAST_LIO)
- ✓ 原版代码: [hku-mars/FAST\\_LIO: A computationally efficient and robust LiDAR-inertial odometry \(LIO\) package \(github.com\)](https://github.com/hku-mars/FAST_LIO)
- ✓ 安装编译参考: [Arafat-ninja/FAST\\_LIO: 仅用于学习 \(github.com\)](https://github.com/Arafat-ninja/FAST_LIO)

# ● S-FAST-LIO安装编译

## 1.1 Ubuntu and ROS

Ubuntu >= 16.04.

## 1.2. PCL & Eigen

PCL >= 1.8, Eigen >= 3.3.4.

## 1.3. livox\_ros\_driver

Follow [livox\\_ros\\_driver Installation](#).

Source: Add the line `source $Livox_ros_driver_dir$/devel/setup.bash` to the end of file `~/.bashrc`, where `$Livox_ros_driver_dir$` is the directory of the livox ros driver workspace (should be the `ws_livox` directory if you completely followed the livox official document).

## 1.4. Sophus

We use the old version of Sophus

```
git clone https://github.com/strasdat/Sophus.git
cd Sophus
git checkout a621ff
mkdir build
cd build
cmake ../ -DUSE_BASIC_LOGGING=ON
make
sudo make install
```

## 2. Build S-FAST\_LIO

Clone the repository and catkin\_make:

```
cd ~/catkin_ws/src
git clone https://github.com/zlwang7/S-FAST_LIO.git
cd ../
catkin_make
source ~/catkin_ws/devel/setup.bash
```

# ● S-FAST-LIO运行与参数调整

## ➤ Launch文件

- ❑ 点云下采样参数
- ❑ Config路径

Run:

```
roslaunch sfast_lio mapping_avia.launch  
rosbag play YOUR_DOWNLOADED.bag
```

## ➤ Yaml文件

- ❑ Common: 话题名称, 时间同步
- ❑ Preprocess: 雷达类型, 线数, 盲区大小
- ❑ Mapping: acc、gyr初始协方差, ba、bw初始协方差
- ❑ Mapping: 是否估计外参、外参值
- ❑ Publish: 是否发布指定消息
- ❑ Pcd\_save: 是否保存地图, 以及保存地图间隔

## ● 录制并运行自己的数据集

- 启动雷达、IMU驱动，发布ROS消息
- `rosv bag record -a` 或 `rosv bag record /topic_name` 录制指定话题
- 在对应型号的yaml中改话题名称

### ◆ 注意事项：

- ❑ 确保时间同步正确，建议采用ROS的系统时间
- ❑ 旋转外参需要标定，或安装在同一角度
- ❑ 平移外参可以手工测量，影响不大
- ❑ IMU的频率建议开高，200hz以上最好
- ❑ 开始的几秒钟不要移动设备，让IMU初始化

谢谢



## 5. FAST-LIO整体框架

### Abstract

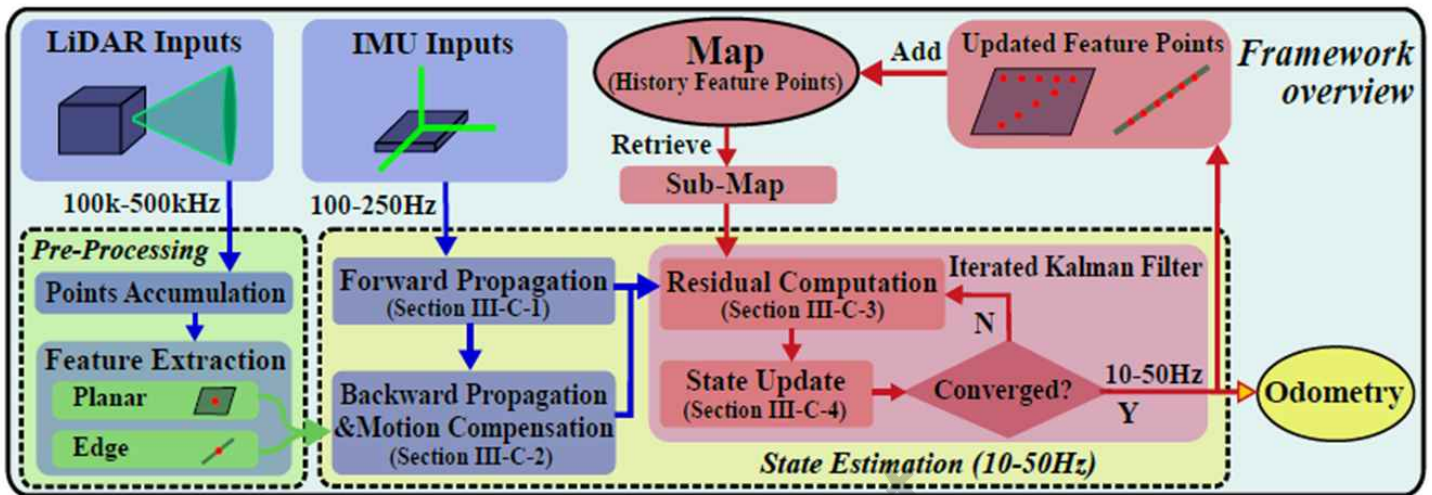
- 高计算效率、高鲁棒性的雷达里程计
- 紧耦合迭代扩展卡尔曼滤波
- 新卡尔曼增益计算公式
- 1200特征点 25ms

### Introduction

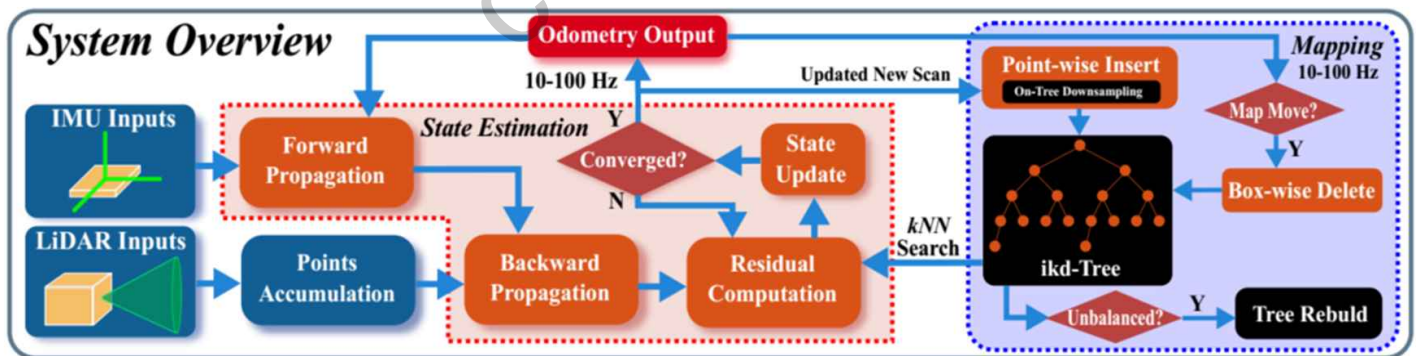
- 视觉传感器对光照敏感、雷达（旋转式）体积大成本高，不适用于小型机器人
- 固态雷达低成本、重量轻
- 固态雷达带来的新挑战：
  - 1) 特征少的场景易退化，小FOV雷达更明显
  - 2) 数千个特征点+IMU数据导致计算量太大
  - 3) 激光点采样时间不同，运动失真
- 本文提出了：
  - 1) 迭代卡尔曼滤波融合雷达和IMU数据，反向传播来补偿运动失真

- 2) 新卡尔曼增益计算公式，减少计算量
- 3) 实验表明能在四旋翼机载计算机实时运行

## Framework Overview



- 雷达点云预处理：点云累积、特征提取
- IMU数据：前向传播、反向传播、运动补偿
- 状态估计模块：残差计算、迭代更新
- 发布里程计、更新地图



## Fast-lio2与Fast-lio1的区别

- 省略了特征提取模块
- 只计算面点残差
- 添加了对外参的优化
- 添加ikdtree数据结构

# 广义运算符

广义运算符的定义：

$$\begin{aligned} \boxplus : \mathcal{M} \times \mathbb{R}^n &\rightarrow \mathcal{M}; & \boxminus : \mathcal{M} \times \mathcal{M} &\rightarrow \mathbb{R}^n \\ \mathcal{M} = SO(3) : \mathbf{R} \boxplus \mathbf{r} &= \mathbf{R} \text{Exp}(\mathbf{r}); & \mathbf{R}_1 \boxminus \mathbf{R}_2 &= \log(\mathbf{R}_2^T \mathbf{R}_1) \\ \mathcal{M} = \mathbb{R}^n : \mathbf{a} \boxplus \mathbf{b} &= \mathbf{a} + \mathbf{b}; & \mathbf{a} \boxminus \mathbf{b} &= \mathbf{a} - \mathbf{b} \end{aligned} \quad (1)$$

这里的  $\mathcal{M}$  表示一种n维的流形。

复合运算的定义：

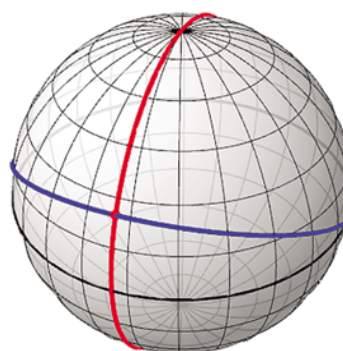
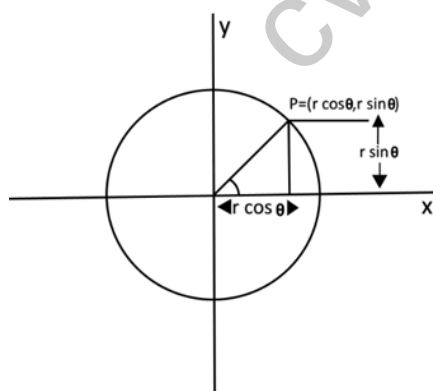
$$\begin{bmatrix} \mathbf{R} \\ \mathbf{a} \end{bmatrix} \boxplus \begin{bmatrix} \mathbf{r} \\ \mathbf{b} \end{bmatrix} = \begin{bmatrix} \mathbf{R} \boxplus \mathbf{r} \\ \mathbf{a} + \mathbf{b} \end{bmatrix}; \quad \begin{bmatrix} \mathbf{R}_1 \\ \mathbf{a} \end{bmatrix} \boxminus \begin{bmatrix} \mathbf{R}_2 \\ \mathbf{b} \end{bmatrix} = \begin{bmatrix} \mathbf{R}_1 \boxminus \mathbf{R}_2 \\ \mathbf{a} - \mathbf{b} \end{bmatrix} \quad (2)$$

以及易于验证的结论：

$$(\mathbf{x} \boxplus \mathbf{u}) \boxminus \mathbf{x} = \mathbf{u}; \quad \mathbf{x} \boxplus (\mathbf{y} \boxminus \mathbf{x}) = \mathbf{y}; \quad \forall \mathbf{x}, \mathbf{y} \in \mathcal{M}, \forall \mathbf{u} \in \mathbb{R}^n. \quad (3)$$

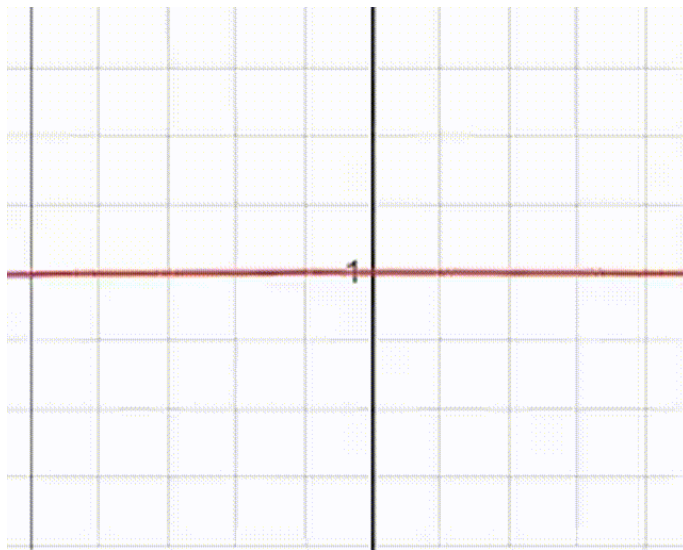
## 流形(Manifold)

- 局部具有欧几里得空间性质的空间,在数学中用于描述几何形体



二维空间的圆是一个一维流形,三维空间的球面是一个二维流形

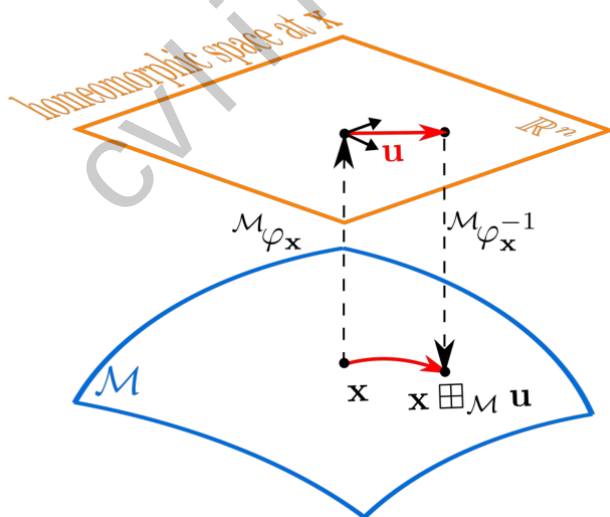
- 我们所能观察到的数据实际上是由一个低维流形映射到高维空间上的，即这些数据所在的空间是“嵌入在高维空间的低维流形”
- 流形是一个“空间”，而不是一个“形状”



局部看是欧式空间（直线），整体看是非欧（曲线）

## 流形的局部同胚(locally homeomorphic)

设流形为  $\mathcal{M}$ , 对于  $\forall x \in \mathcal{M}$  和一个包含  $x$  的开子集  $U \subset \mathcal{M}$ , 存在一个双射函数(又叫同胚)  $\phi$ , 它可以把  $U$  中的点映射到  $\mathbb{R}^n$  上的一个开子集。



$\mathcal{M}_{\varphi_x}$  是点  $x$  附近的同胚(双射函数)。广义的  $\boxplus$  和  $\boxminus$  是这样定义的：

$$\begin{aligned} \boxplus : \mathcal{M} \times \mathbb{R}^n &\rightarrow \mathcal{M} & \mathbf{x} \boxplus_{\mathcal{M}} \mathbf{u} &= \mathcal{M}_{\varphi_x^{-1}}(\mathbf{u}) \\ \boxminus : \mathcal{M} \times \mathcal{M} &\rightarrow \mathbb{R}^n & \mathbf{y} \boxminus_{\mathcal{M}} \mathbf{x} &= \mathcal{M}_{\varphi_x}(\mathbf{y}) \end{aligned} \quad (4)$$

其中  $\forall \mathbf{x}, \mathbf{y} \in \mathcal{M}, \mathbf{u} \in \mathbb{R}^n$ 。

当  $\mathcal{M} = SO(3)$  时：

$$\mathbf{x} \boxplus_{\mathcal{M}} \mathbf{u} = \mathbf{x} \cdot \text{Exp}(\mathbf{u}); \quad \mathbf{y} \boxminus_{\mathcal{M}} \mathbf{x} = \log(\mathbf{x}^{-1} \cdot \mathbf{y}) \quad (5)$$

此时  $\mathcal{M}_{\varphi_{\mathbf{x}}}$  是对数映射， $\mathcal{M}_{\varphi_{\mathbf{x}}^{-1}}$  是指数映射。

cvlife.net



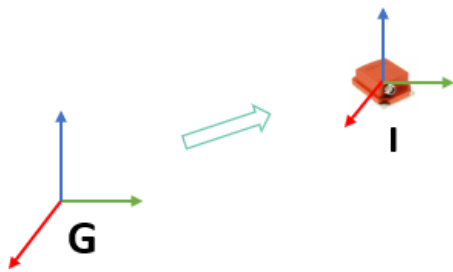
## 6. IMU连续模型与离散模型

### IMU连续模型

$$\begin{aligned} {}^G\dot{\mathbf{p}}_I &= {}^G\mathbf{v}_I, {}^G\dot{\mathbf{v}}_I = {}^G\mathbf{R}_I (\mathbf{a}_m - \mathbf{b}_a - \mathbf{n}_a) + {}^G\mathbf{g}, {}^G\dot{\mathbf{g}} = \mathbf{0} \\ {}^G\dot{\mathbf{R}}_I &= {}^G\mathbf{R}_I [\boldsymbol{\omega}_m - \mathbf{b}_\omega - \mathbf{n}_\omega]_{\wedge}, \dot{\mathbf{b}}_\omega = \mathbf{n}_{b\omega}, \dot{\mathbf{b}}_a = \mathbf{n}_{ba} \end{aligned} \quad (1)$$

- 加速度g不变，导数为零
- bias服从随机游走模型，其导数是高斯的

回顾测量模型（测量值=真值+零偏+噪声），以及IMU连续运动模型：



➤ 在G系下,连续运动的导数:

$$\dot{\mathbf{R}} = \mathbf{R} \boldsymbol{\omega}^\wedge$$

$$\dot{\mathbf{v}} = \mathbf{a}$$

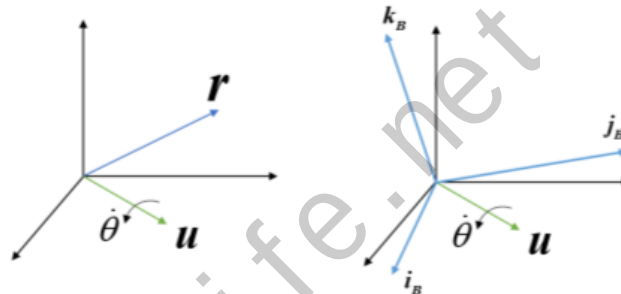
$$\dot{\mathbf{p}} = \mathbf{v}$$

➤ 测量模型:

$$\mathbf{a}_m = \mathbf{a} + \mathbf{b}_a + \mathbf{n}_a - {}^I \mathbf{R}_G {}^G \mathbf{g}$$

$$\boldsymbol{\omega}_m = \boldsymbol{\omega} + \mathbf{b}_w + \mathbf{n}_w$$

$$\dot{\mathbf{b}}_w = \mathbf{n}_{bw}, \dot{\mathbf{b}}_a = \mathbf{n}_{ba}$$



考虑一个从原点出发的向量 $\mathbf{r}$ 绕单位轴 $\mathbf{u}$ 旋转, 角速度为 $\boldsymbol{\omega} = \dot{\theta} \mathbf{u}$ , 角速度大小为 $\dot{\theta}$ , 则:

$$\frac{d\mathbf{r}}{dt} = \boldsymbol{\omega} \times \mathbf{r}$$

坐标系绕单位轴 $\mathbf{u}$ 旋转, 其三个轴的导数:

$$\frac{d\mathbf{i}_B}{dt} = \boldsymbol{\omega} \times \mathbf{i}_B, \frac{d\mathbf{j}_B}{dt} = \boldsymbol{\omega} \times \mathbf{j}_B, \frac{d\mathbf{k}_B}{dt} = \boldsymbol{\omega} \times \mathbf{k}_B$$

$[\mathbf{i}_B, \mathbf{j}_B, \mathbf{k}_B]$  实际上就是坐标系  $\{B\}$  相对于原坐标系的旋转矩阵  $\mathbf{R}$

$$\dot{\mathbf{R}} = [\boldsymbol{\omega} \times \mathbf{i}_B \quad \boldsymbol{\omega} \times \mathbf{j}_B \quad \boldsymbol{\omega} \times \mathbf{k}_B] = \boldsymbol{\omega} \times \mathbf{R} = \boldsymbol{\omega}^\wedge \mathbf{R}$$

这里的 $\boldsymbol{\omega}$ 是原坐标系下的, 根据  $(R\mathbf{p})^\wedge = R\mathbf{p}^\wedge R^T$

$${}^G \dot{\mathbf{R}}_I = ({}^G \mathbf{R}_I {}^I \boldsymbol{\omega})^\wedge {}^G \mathbf{R}_I = {}^G \mathbf{R}_I ({}^I \boldsymbol{\omega})^\wedge$$

## IMU离散模型

状态量的定义:

$$\mathbf{x} \doteq \begin{bmatrix} {}^G\mathbf{R}_I^T & {}^G\mathbf{p}_I^T & {}^G\mathbf{v}_I^T & \mathbf{b}_\omega^T & \mathbf{b}_a^T & {}^G\mathbf{g}^T \end{bmatrix}^T \in \mathcal{M} \quad (2)$$

其中  $\mathcal{M} = SO(3) \times \mathbb{R}^{15}$ ,  $\dim(\mathcal{M}) = 18$ 。状态量即为所有的待估计变量，共18维。可以看出我们要实时估计的是一个18维的量 $\mathbf{x}$ ,它包含位置，姿态，速度以及IMU的参数。因此它是一个紧耦合的框架。

输入 $\mathbf{u}$ 即加速度测量值：

$$\mathbf{u} \doteq \begin{bmatrix} \boldsymbol{\omega}_m^T & \mathbf{a}_m^T \end{bmatrix}^T \quad (3)$$

$\mathbf{w}$ 为所有的噪声量：

$$\mathbf{w} \doteq \begin{bmatrix} \mathbf{n}_\omega^T & \mathbf{n}_a^T & \mathbf{n}_{b\omega}^T & \mathbf{n}_{ba}^T \end{bmatrix}^T \quad (4)$$

包含测量噪声（前两项）和bias随机游走噪声（后两项）。

**论文中离散模型：**

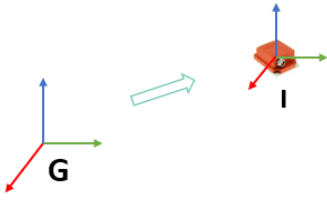
$$\mathbf{x}_{i+1} = \mathbf{x}_i \boxplus (\Delta t \mathbf{f}(\mathbf{x}_i, \mathbf{u}_i, \mathbf{w}_i)) \quad (5)$$

其中 $i$ 为IMU测量的索引。其中：

$$\mathbf{f}(\mathbf{x}_i, \mathbf{u}_i, \mathbf{w}_i) = \begin{bmatrix} \boldsymbol{\omega}_{m_i} - \mathbf{b}_{\omega_i} - \mathbf{n}_{\omega_i} \\ {}^G\mathbf{v}_{I_i} \\ \mathbf{R}_{I_i}(\mathbf{a}_{m_i} - \mathbf{b}_{a_i} - \mathbf{n}_{a_i}) + {}^G\mathbf{g}_i \\ \mathbf{n}_{b\omega_i} \\ \mathbf{n}_{ba} \\ \mathbf{0}_{3 \times 1} \end{bmatrix} \quad (6)$$

回顾IMU离散模型：

## IMU离散模型



➤ 测量模型：

$$\mathbf{a}_m = \mathbf{a} + \mathbf{b}_a + \mathbf{n}_a - {}^I \mathbf{R}_G {}^G \mathbf{g}$$

$$\boldsymbol{\omega}_m = \boldsymbol{\omega} + \mathbf{b}_w + \mathbf{n}_w$$

$$\dot{\mathbf{b}}_w = \mathbf{n}_{bw}, \dot{\mathbf{b}}_a = \mathbf{n}_{ba}$$

➤ 离散模型：

$$\mathbf{R}(t + \Delta t) = \mathbf{R}(t) \text{Exp}(\boldsymbol{\omega}(t) \Delta t)$$

$$\mathbf{v}(t + \Delta t) = \mathbf{v}(t) + \mathbf{a}(t) \Delta t$$

$$\mathbf{p}(t + \Delta t) = \mathbf{p}(t) + \mathbf{v}(t) \Delta t + \frac{1}{2} \mathbf{a}(t) \Delta t^2$$

这里的a是G系下的a

➤ 将测量模型代入：

$$\mathbf{R}(t + \Delta t) = \mathbf{R}(t) \text{Exp}[(\boldsymbol{\omega}_m - \mathbf{b}_w - \mathbf{n}_w) \Delta t]$$

$$\mathbf{v}(t + \Delta t) = \mathbf{v}(t) + [\mathbf{R}(t)(\mathbf{a}_m - \mathbf{b}_a - \mathbf{n}_a) + {}^G \mathbf{g}] \Delta t$$

$$\mathbf{p}(t + \Delta t) = \mathbf{p}(t) + \mathbf{v}(t) \Delta t$$

IMU积分：每收到一帧IMU数据执行一次上式

$$\dot{\mathbf{R}} = \mathbf{R} \boldsymbol{\omega}^\wedge$$

$$\dot{\mathbf{v}} = \mathbf{a}$$

$$\dot{\mathbf{p}} = \mathbf{v}$$

即：

$$\mathbf{R}_{i+1} = \mathbf{R}_i \text{Exp}((\boldsymbol{\omega}_{m_i} - \mathbf{b}_{w_i} - \mathbf{n}_{w_i}) \Delta t) \quad (7)$$

$$\mathbf{p}_{i+1} = \mathbf{p}_i + {}^G \mathbf{v}_{I_i} \Delta t \quad (8)$$

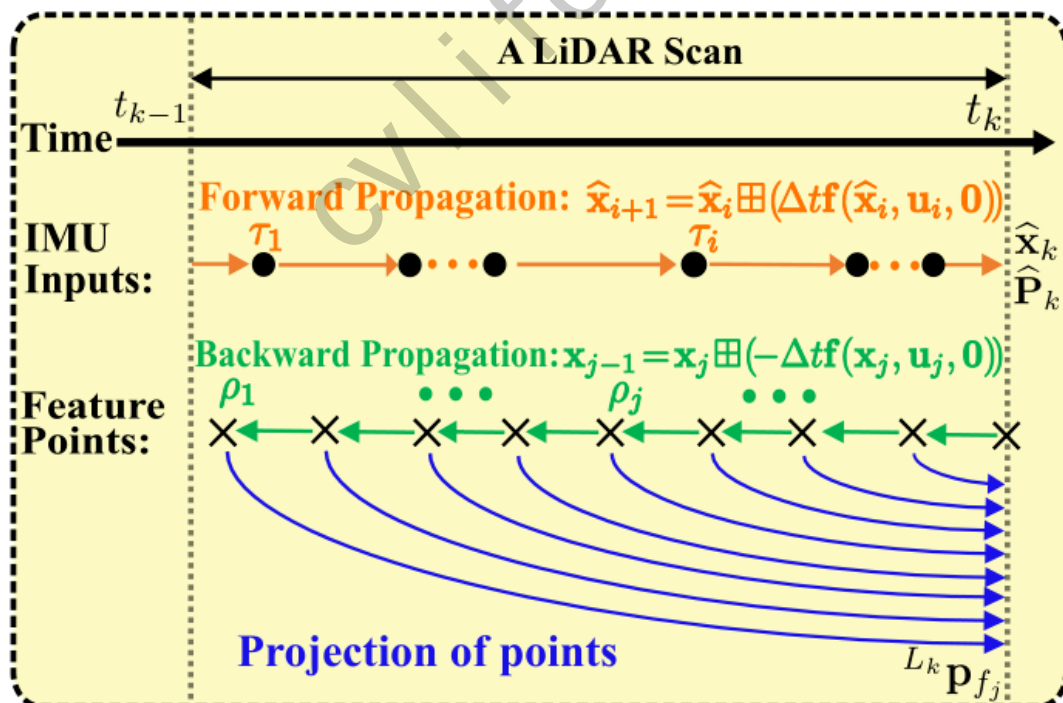
$$\mathbf{v}_{i+1} = \mathbf{v}_i + (\mathbf{R}_{I_i} (\mathbf{a}_{m_i} - \mathbf{b}_{a_i} - \mathbf{n}_{a_i}) + {}^G \mathbf{g}_i) \Delta t \quad (9)$$

$$\mathbf{b}_{w_{i+1}} = \mathbf{b}_{w_i} + \mathbf{n}_{bw_i} \Delta t, \mathbf{b}_{a_{i+1}} = \mathbf{b}_{a_i} + \mathbf{n}_{ba_i} \Delta t \quad (10)$$



## 7.点云预处理和前向传播

### 点云预处理



- 原始点的采集频率很高(200kHz),因此通常累计一定时间(20ms,即50Hz)的点作为一次scan
- 从原始点中提取平面点和边缘点(LOAM),设特征点的总数量为 $m$ ,  $m$ , 每个特征点的时间戳为  $\rho_j$ , 并定义每个IMU 帧的时间戳为  $\tau_i$ , 如图
- 最后一个特征点是一次scan的终点, 即  $\rho_m = t_k$ , 但IMU测量值不与起点或终点对齐

# 前向传播

明确文中的符号的定义：

- $x$  表示真值
- $\bar{x}$  表示滤波算法结束后得到的最优估计
- $\hat{x}$  表示滤波算法过程中得到的估计值
- $\tilde{x}$  表示误差

误差量的定义：

$$\tilde{\mathbf{x}}_{k-1} \doteq \mathbf{x}_{k-1} \ominus \bar{\mathbf{x}}_{k-1} = \begin{bmatrix} \delta \boldsymbol{\theta}^T & {}^G \tilde{\mathbf{p}}_I^T & {}^G \tilde{\mathbf{v}}_I^T & \tilde{\mathbf{b}}_\omega^T & \tilde{\mathbf{b}}_a^T & {}^G \tilde{\mathbf{g}}^T \end{bmatrix}^T \quad (1)$$

- 前向传播目的1：通过IMU积分计算一个粗略的状态量  $\hat{x}_k$ ，用于后续的反向传播来补偿运动失真

$$\hat{\mathbf{x}}_{i+1} = \hat{\mathbf{x}}_i \boxplus (\Delta t \mathbf{f}(\hat{\mathbf{x}}_i, \mathbf{u}_i, \mathbf{0})); \hat{\mathbf{x}}_0 = \bar{\mathbf{x}}_{k-1} \quad (2)$$

$\Delta t = \tau_{i+1} - \tau_i$ , 噪声量  $\mathbf{w}_i = \mathbf{0}$  是因为我们并不知道噪声的实际大小，因此在传播过程中设为零。

- 前向传播目的2：传播误差量，并计算对应的协方差矩阵

$$\begin{aligned} \tilde{\mathbf{x}}_{i+1} &= \mathbf{x}_{i+1} \ominus \hat{\mathbf{x}}_{i+1} \\ &= (\mathbf{x}_i \boxplus \Delta t \mathbf{f}(\mathbf{x}_i, \mathbf{u}_i, \mathbf{w}_i)) \ominus (\hat{\mathbf{x}}_i \boxplus \Delta t \mathbf{f}(\hat{\mathbf{x}}_i, \mathbf{u}_i, \mathbf{0})) \\ &\simeq \mathbf{F}_{\tilde{\mathbf{x}}} \tilde{\mathbf{x}}_i + \mathbf{F}_{\mathbf{w}} \mathbf{w}_i. \end{aligned} \quad (3)$$

假设白噪声  $\mathbf{w}$  的协方差矩阵为  $\mathbf{Q}$ ，则可以按照下式传播协方差矩阵：

$$\hat{\mathbf{P}}_{i+1} = \mathbf{F}_{\tilde{\mathbf{x}}} \hat{\mathbf{P}}_i \mathbf{F}_{\tilde{\mathbf{x}}}^T + \mathbf{F}_{\mathbf{w}} \mathbf{Q} \mathbf{F}_{\mathbf{w}}^T; \hat{\mathbf{P}}_0 = \bar{\mathbf{P}}_{k-1} \quad (4)$$

其中

$$\begin{aligned} \mathbf{A}(\mathbf{u})^{-1} &= \mathbf{I} - \frac{1}{2} [\mathbf{u}]_{\wedge} + (1 - \alpha(\|\mathbf{u}\|)) \frac{[\mathbf{u}]_{\Delta}^2}{\|\mathbf{u}\|^2} \\ \alpha(m) &= \frac{m}{2} \cot\left(\frac{m}{2}\right) = \frac{m \cos(m/2)}{2 \sin(m/2)} \end{aligned} \quad (5)$$

这个公式代入性质  $u^{\wedge^2} = uu^T - I$  (u模长为1) 即BCH近似雅可比。

# 前向传播公式推导

## 已知条件和求解量

- $\tilde{\mathbf{x}}_{i+1} = \mathbf{x}_{i+1} \boxminus \hat{\mathbf{x}}_{i+1} = (\mathbf{x}_i \boxplus \Delta t \mathbf{f}(\mathbf{x}_i, \mathbf{u}_i, \mathbf{w}_i)) \boxminus (\hat{\mathbf{x}}_i \boxplus \Delta t \mathbf{f}(\hat{\mathbf{x}}_i, \mathbf{u}_i, \mathbf{0}))$
- $\mathbf{x}_i = \hat{\mathbf{x}}_i \boxplus \tilde{\mathbf{x}}_i$
- 定义  $\mathbf{g}(\tilde{\mathbf{x}}_i, \mathbf{w}_i) = \mathbf{f}(\mathbf{x}_i, \mathbf{u}_i, \mathbf{w}_i) \Delta t = \mathbf{f}(\hat{\mathbf{x}}_i \boxplus \tilde{\mathbf{x}}_i, \mathbf{u}_i, \mathbf{w}_i) \Delta t$

第一个式子可以改写为：

$$\tilde{\mathbf{x}}_{i+1} = [(\hat{\mathbf{x}}_i \boxplus \tilde{\mathbf{x}}_i) \boxplus \mathbf{g}(\tilde{\mathbf{x}}_i, \mathbf{w}_i)] \boxminus [\hat{\mathbf{x}}_i \boxplus \mathbf{g}(\mathbf{0}, \mathbf{0})] \stackrel{def}{=} \mathbf{G}[\tilde{\mathbf{x}}_i, \mathbf{g}(\tilde{\mathbf{x}}_i, \mathbf{w}_i)] \quad (6)$$

以上内容见原文附录。为了简化，在后续推导中我们省去上式中烦人的下标*i*。

我们要求的是： $\mathbf{G}(\tilde{\mathbf{x}}, \mathbf{g}(\tilde{\mathbf{x}}, \mathbf{w}))$  对  $\tilde{\mathbf{x}}$  和  $\mathbf{w}$  的偏导数（在  $\tilde{\mathbf{x}} = \mathbf{w} = \mathbf{0}$  附近），即复合函数求偏导。

对  $\tilde{\mathbf{x}}$  求偏导时， $\mathbf{w} = \mathbf{0}$ ：

$$F_{\tilde{\mathbf{x}}} = \frac{\partial G(\tilde{\mathbf{x}}, g(\mathbf{0}, \mathbf{0}))}{\partial \tilde{\mathbf{x}}} + \frac{\partial G(\mathbf{0}, g(\tilde{\mathbf{x}}, \mathbf{0}))}{\partial g(\tilde{\mathbf{x}}, \mathbf{0})} \frac{\partial g(\tilde{\mathbf{x}}, \mathbf{0})}{\partial \tilde{\mathbf{x}}} \bigg|_{\tilde{\mathbf{x}}=\mathbf{0}} \stackrel{simplify}{=} \frac{\partial G}{\partial \tilde{\mathbf{x}}} + \frac{\partial G}{\partial g} \frac{\partial g}{\partial \tilde{\mathbf{x}}} \bigg|_{\tilde{\mathbf{x}}=\mathbf{0}} \quad (7)$$

对  $\mathbf{w}$  求偏导时， $\tilde{\mathbf{x}} = \mathbf{0}$ ：

$$F_{\mathbf{w}} = \frac{\partial G(\mathbf{0}, g(\mathbf{0}, \mathbf{w}))}{\partial g(\mathbf{0}, \mathbf{w})} \frac{\partial g(\mathbf{0}, \mathbf{w})}{\partial \mathbf{w}} \bigg|_{\mathbf{w}=\mathbf{0}} \stackrel{simplify}{=} \frac{\partial G}{\partial g_{\mathbf{w}}} \frac{\partial g}{\partial \mathbf{w}} \bigg|_{\mathbf{w}=\mathbf{0}} \quad (8)$$

## 关于流形偏导数的结论

接下来，我们先考虑这样一个流形的偏导数。对于流形  $\mathbf{E} = ((\mathbf{a} \boxplus \mathbf{b}) \boxplus \mathbf{c}) \boxminus \mathbf{d}$ ，有以下结论：

$$\frac{\partial \mathbf{E}}{\partial \mathbf{b}} = \frac{\partial \mathbf{E}}{\partial \mathbf{c}} = \mathbf{I}_{n \times n}, \quad a, b, c, d \in \mathbb{R}^n \quad (9)$$

$$\frac{\partial \mathbf{E}}{\partial \mathbf{b}} = \mathbf{A}(\mathbf{E})^{-T} \mathbf{E} \exp(-\mathbf{c}) \mathbf{A}(\mathbf{b})^T, \quad \frac{\partial \mathbf{E}}{\partial \mathbf{c}} = \mathbf{A}(\mathbf{E})^{-T} \mathbf{A}(\mathbf{c})^T, \quad a, d \in SO(3), b, c \in \mathbb{R}^3 \quad (10)$$

对于  $\frac{\partial \mathbf{E}}{\partial \mathbf{b}}$  :

$$\begin{aligned}
E &= ((a \boxplus b) \boxplus c) \boxminus d = \text{Log}(d^{-1} \cdot (a \boxplus b) \boxplus c)) \\
&\rightarrow \text{Exp}(E) = d^{-1} \cdot a \cdot \text{Exp}(b) \cdot \text{Exp}(c) \\
&\rightarrow \text{Exp}(E + \Delta E) = d^{-1} \cdot a \cdot \text{Exp}(b + \Delta b) \cdot \text{Exp}(c) \\
&\xrightarrow{BCH} \text{Exp}(E) \text{Exp}(A(E)^T \Delta E) = d^{-1} \cdot a \cdot \text{Exp}(b) \text{Exp}(A(b)^T \Delta b) \cdot \text{Exp}(c) \\
&\rightarrow \text{Exp}(A(E)^T \Delta E) = \text{Exp}(c)^T \text{Exp}(A(b)^T \Delta b) \cdot \text{Exp}(c) \\
&\rightarrow \text{Exp}(A(E)^T \Delta E) \xrightarrow{\text{Property}} \text{Exp}(\text{Exp}(-c) A(b)^T \Delta b) \\
&\rightarrow \frac{\Delta E}{\Delta b} = A(E)^{-T} \text{Exp}(-c) A(b)^T
\end{aligned} \tag{11}$$

其中第四行用了BCH近似公式，倒数第二行用了SO(3)上矩阵的伴随性质：

$$R \exp(p^\wedge) R^T = \exp((Rp)^\wedge).$$

对于  $\frac{\partial \mathbf{E}}{\partial \mathbf{c}}$  :

$$\begin{aligned}
&\rightarrow \text{Exp}(E + \Delta E) = d^{-1} \cdot a \cdot \text{Exp}(b) \cdot \text{Exp}(c + \Delta c) \\
&\xrightarrow{BCH} \text{Exp}(E) \text{Exp}(A(E)^T \Delta E) = d^{-1} \cdot a \cdot \text{Exp}(b) \cdot \text{Exp}(c) \text{Exp}(A(c)^T \Delta c) \\
&\rightarrow \text{Exp}(A(E)^T \Delta E) = \text{Exp}(A(c)^T \Delta c) \\
&\rightarrow \frac{\Delta E}{\Delta c} = A(E)^{-T} A(c)^T
\end{aligned} \tag{12}$$

实际上  $\mathbf{G}$  就是形如  $((\mathbf{a} \boxplus \mathbf{b}) \boxplus \mathbf{c}) \boxminus \mathbf{d}$  的形式，因为  $\mathbf{G} = (\hat{\mathbf{x}} \boxplus \tilde{\mathbf{x}}) \boxplus \mathbf{g}(\tilde{\mathbf{x}}, \mathbf{w}) \boxminus (\hat{\mathbf{x}} \boxplus \mathbf{g}(\mathbf{0}, \mathbf{0}))$ 。

## 代入结论求解

$$\text{由 } F_{\tilde{\mathbf{x}}} = \frac{\partial G}{\partial \tilde{\mathbf{x}}} + \frac{\partial G}{\partial \mathbf{g}} \frac{\partial \mathbf{g}}{\partial \tilde{\mathbf{x}}} \Big|_{\tilde{\mathbf{x}}=\mathbf{0}} :$$

$$\begin{aligned}
\left. \frac{\partial G}{\partial \tilde{x}} \right|_{\tilde{x}=0} &= \begin{bmatrix} \mathbf{A}(\mathbf{0})^{-1} \mathbf{Exp}(-\mathbf{g}(\mathbf{0}, \mathbf{0})) \mathbf{A}(\mathbf{0})^T & 0 \\ 0 & \mathbf{I}_{15 \times 15} \end{bmatrix} \\
&= \begin{bmatrix} \mathbf{Exp}(-\mathbf{f}(\hat{\mathbf{x}}, \mathbf{u}, \mathbf{0}) \Delta t) & 0 \\ 0 & \mathbf{I}_{15 \times 15} \end{bmatrix} \\
&= \begin{bmatrix} \mathbf{Exp}(-(\omega_m - \hat{\mathbf{b}}_\omega) \Delta t) & 0 \\ 0 & \mathbf{I}_{15 \times 15} \end{bmatrix}
\end{aligned} \tag{13}$$

其中，当  $\tilde{x} = 0$  时， $G = 0$  且  $A(0) = I$ 。

同理：

$$\begin{aligned}
\left. \frac{\partial G}{\partial g} \right|_{\tilde{x}=0} &= \begin{bmatrix} \mathbf{A}(\mathbf{0})^{-1} \mathbf{A}(\mathbf{g}(\mathbf{0}, \mathbf{0}))^T & 0 \\ 0 & \mathbf{I}_{15 \times 15} \end{bmatrix} \\
&= \begin{bmatrix} \mathbf{A}((\omega_m - \hat{\mathbf{b}}_\omega) \Delta t) & 0 \\ 0 & \mathbf{I}_{15 \times 15} \end{bmatrix}
\end{aligned} \tag{14}$$

关于  $\frac{\partial g}{\partial \tilde{x}}$ ，由于误差状态的定义  $\tilde{\mathbf{x}} \doteq \mathbf{x} \ominus \hat{\mathbf{x}} = [\delta\theta^T \quad {}^G\tilde{\mathbf{p}}_I^T \quad {}^G\tilde{\mathbf{v}}_I^T \quad \tilde{\mathbf{b}}_\omega^T \quad \tilde{\mathbf{b}}_a^T \quad {}^G\tilde{\mathbf{g}}^T]^T$ ， $\mathbf{x}_i = \hat{\mathbf{x}}_i \oplus \tilde{\mathbf{x}}_i$ ，和  $\mathbf{w} \doteq [\mathbf{n}_\omega^T \quad \mathbf{n}_a^T \quad \mathbf{n}_{b\omega}^T \quad \mathbf{n}_{ba}^T]^T$ ，则：

$$\begin{aligned}
\mathbf{g}(\tilde{\mathbf{x}}, \mathbf{w}) &= \mathbf{f}(\mathbf{x}, \mathbf{u}, \mathbf{w}) \Delta t = \begin{bmatrix} \omega_m - \mathbf{b}_\omega - \mathbf{n}_\omega \\ {}^G\tilde{\mathbf{v}}_I \\ \mathbf{R}_I(\mathbf{a}_m - \mathbf{b}_a - \mathbf{n}_a) + {}^G\mathbf{g} \\ \mathbf{n}_{b\omega} \\ \mathbf{n}_{ba} \\ \mathbf{0}_{3 \times 1} \end{bmatrix} \Delta t \\
&= \begin{bmatrix} \omega_m - \hat{\mathbf{b}}_\omega - \tilde{\mathbf{b}}_\omega - \mathbf{n}_\omega \\ {}^G\hat{\mathbf{v}}_I + {}^G\tilde{\mathbf{v}}_I \\ \hat{\mathbf{R}}_I \mathbf{Exp}(\delta\theta^T) (\mathbf{a}_m - \hat{\mathbf{b}}_a - \tilde{\mathbf{b}}_a - \mathbf{n}_a) + {}^G\mathbf{g} \\ \mathbf{n}_{b\omega} \\ \mathbf{n}_{ba} \\ \mathbf{0}_{3 \times 1} \end{bmatrix} \Delta t
\end{aligned} \tag{15}$$

则

$$\left. \frac{\partial g(\tilde{\mathbf{x}}, 0)}{\partial \tilde{x}} \right|_{\tilde{x}=0} = \begin{pmatrix} 0 & 0 & 0 & -\mathbf{I}_{3 \times 3} \Delta t & 0 & 0 \\ 0 & 0 & \mathbf{I}_{3 \times 3} \Delta t & 0 & 0 & 0 \\ -\hat{\mathbf{R}}_I(\mathbf{a}_m - \hat{\mathbf{b}}_a)^T \Delta t & 0 & 0 & 0 & -\hat{\mathbf{R}}_I \Delta t & \mathbf{I}_{3 \times 3} \Delta t \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{pmatrix} \tag{16}$$

其中，第三行关于R的求导利用了右乘扰动模型。至此，根据  $F_{\tilde{x}} = \frac{\partial G}{\partial \tilde{x}} + \frac{\partial G}{\partial g} \frac{\partial g}{\partial \tilde{x}} \Big|_{\tilde{x}=0}$ ，再加上不烦人的下标i，得到：

$$\mathbf{F}_{\tilde{x}} = \begin{pmatrix} \mathbf{Exp}(-(\omega_m - \hat{\mathbf{b}}_\omega)\Delta t) & 0 & 0 & -\mathbf{A}((\omega_m - \hat{\mathbf{b}}_\omega)\Delta t)\Delta t & 0 & 0 \\ 0 & \mathbf{I} & \mathbf{I}\Delta t & 0 & 0 & 0 \\ -{}^G\hat{\mathbf{R}}_{I_i}(\mathbf{a}_m - \hat{\mathbf{b}}_a)^\wedge \Delta t & 0 & \mathbf{I} & 0 & -{}^G\hat{\mathbf{R}}_{I_i}\Delta t & \mathbf{I}\Delta t \\ 0 & 0 & 0 & \mathbf{I} & 0 & 0 \\ 0 & 0 & 0 & 0 & \mathbf{I} & 0 \\ 0 & 0 & 0 & 0 & 0 & \mathbf{I} \end{pmatrix} \quad (17)$$

这和论文中是一致的结论（注意论文中  $\hat{\omega}_i = \omega_{m_i} - \hat{\mathbf{b}}_{\omega_i}$ ,  $\hat{\mathbf{a}}_i = \mathbf{a}_{m_i} - \hat{\mathbf{b}}_{\mathbf{a}_i}$ ）。

同理，根据  $F_w = \frac{\partial G}{\partial g_w} \frac{\partial g}{\partial w} \Big|_{w=0}$ ：

$$\begin{aligned} \frac{\partial G}{\partial g_w} \Big|_{w=0} & \begin{bmatrix} \mathbf{A}(0)^{-1} \mathbf{A}(g(0,0))^T & 0 \\ 0 & \mathbf{I}_{15 \times 15} \end{bmatrix} \\ & = \begin{bmatrix} \mathbf{A}((\omega_m - \hat{\mathbf{b}}_\omega)\Delta t) & 0 \\ 0 & \mathbf{I}_{15 \times 15} \end{bmatrix} \end{aligned} \quad (18)$$

以及：

$$\frac{\partial g}{\partial w} \Big|_{w=0} = \begin{pmatrix} -\mathbf{I}_{3 \times 3} \Delta t & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & -\hat{\mathbf{R}}_I \Delta t & 0 & 0 \\ 0 & 0 & \mathbf{I}_{3 \times 3} & 0 \\ 0 & 0 & 0 & \mathbf{I}_{3 \times 3} \\ 0 & 0 & 0 & 0 \end{pmatrix} \quad (19)$$

加上下标i:

$$F_w = \begin{pmatrix} -\mathbf{A}((\omega_m - \hat{\mathbf{b}}_\omega)\Delta t)\Delta t & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & -\hat{\mathbf{R}}_I \Delta t & 0 & 0 \\ 0 & 0 & \mathbf{I}\Delta t & 0 \\ 0 & 0 & 0 & \mathbf{I}\Delta t \\ 0 & 0 & 0 & 0 \end{pmatrix} \quad (20)$$

我们相当于已经完成了ESKF的运动方程的线性化的推导。

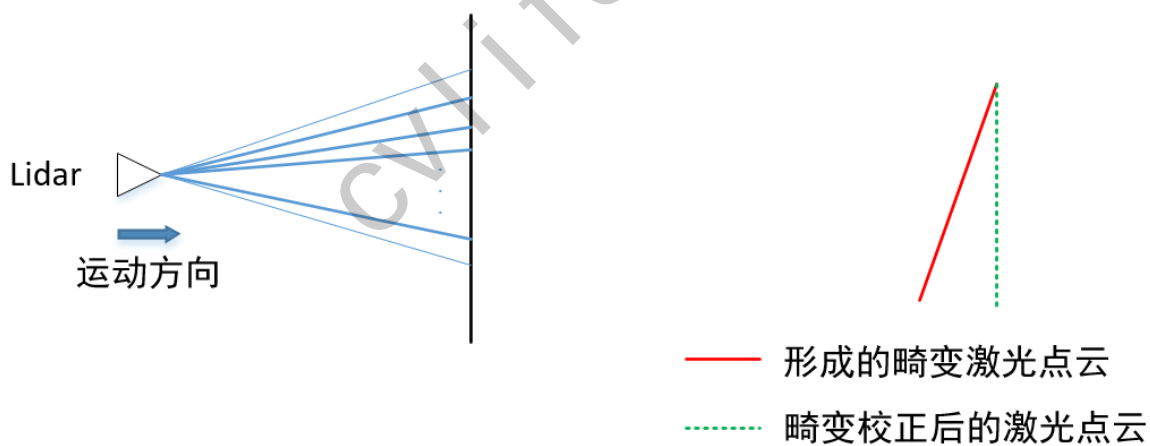
实际上式中的部分量是可以简化的，例如  $-\mathbf{A}((\omega_m - \hat{\mathbf{b}}_\omega)\Delta t)$  可以简化为  $\mathbf{I}$ ，因为  $\Delta t$  足够小。

cvlife.net



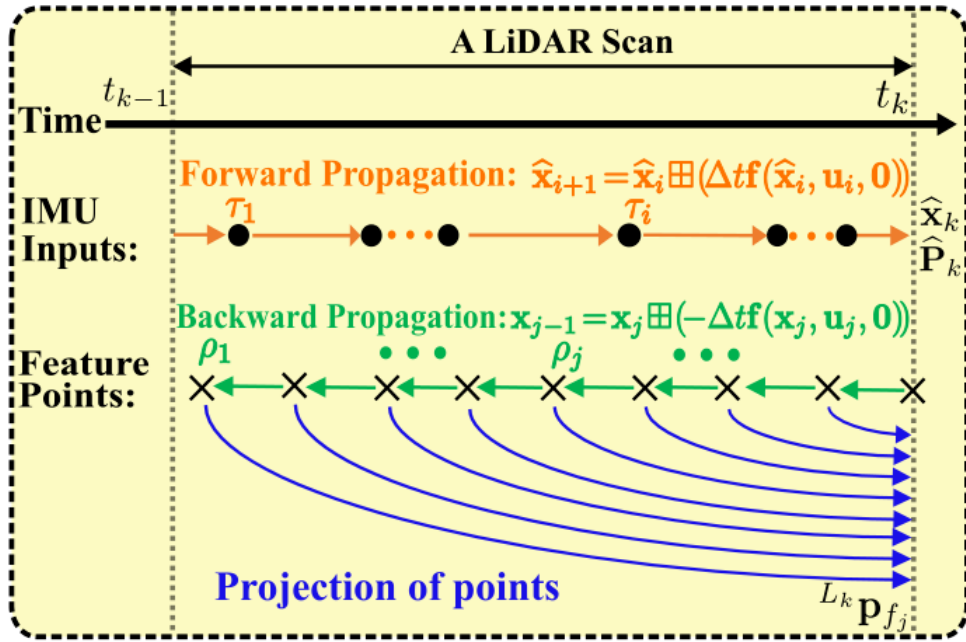
## 8. 反向传播与运动补偿

### 激光雷达运动畸变



- 雷达数据仅有一时间戳，而非某个时刻的数据
- 在这一帧时间内的载体通常会发生运动，这一帧的原点不一致，产生运动畸变
- 在运动速度快时尤为明显

### 反向传播



反向传播传播的是每个特征点的相对状态量，目的是为了雷达点云的运动补偿。

根据IMU积分估计的位姿，把每个点转换到  $t_k$  时刻。

- 通过公式  $\check{\mathbf{x}}_{j-1} = \check{\mathbf{x}}_j \boxplus (-\Delta t \mathbf{f}(\check{\mathbf{x}}_j, \mathbf{u}_j, \mathbf{0}))$  计算相对位姿
- 点的频率大于IMU频率，对于两帧IMU之间的多个特征点，我们以左IMU帧为准

$$\text{回顾f的定义: } \mathbf{f}(\mathbf{x}_i, \mathbf{u}_i, \mathbf{w}_i) = \begin{bmatrix} \boldsymbol{\omega}_{m_i} - \mathbf{b}_{\omega_i} - \mathbf{n}_{\omega_i} \\ G \mathbf{v}_{I_i} \\ \mathbf{R}_{I_i} (\mathbf{a}_{m_i} - \mathbf{b}_{a_i} - \mathbf{n}_{a_i}) + G \mathbf{g}_i \\ \mathbf{n}_{b\omega_i} \\ \mathbf{n}_{ba} \\ \mathbf{0}_{3 \times 1} \end{bmatrix}$$

考虑到传播时噪声量为零，式  $\check{\mathbf{x}}_{j-1} = \check{\mathbf{x}}_j \boxplus (-\Delta t \mathbf{f}(\check{\mathbf{x}}_j, \mathbf{u}_j, \mathbf{0}))$  可以简化为：

$$\begin{aligned} {}^{I_k} \check{\mathbf{p}}_{I_{j-1}} &= {}^{I_k} \check{\mathbf{p}}_{I_j} - {}^{I_k} \check{\mathbf{v}}_{I_j} \Delta t, \quad \text{s.f. } {}^{I_k} \check{\mathbf{p}}_{I_m} = \mathbf{0}; \\ {}^{I_k} \check{\mathbf{v}}_{I_{j-1}} &= {}^{I_k} \check{\mathbf{v}}_{I_j} - {}^{I_k} \check{\mathbf{R}}_{I_j} \left( \mathbf{a}_{m_{i-1}} - \widehat{\mathbf{b}}_{a_k} \right) \Delta t - {}^{I_k} \widehat{\mathbf{g}}_k \Delta t, \quad \text{s.f. } {}^{I_k} \check{\mathbf{v}}_{I_m} = {}^G \widehat{\mathbf{R}}_{I_k}^T {}^G \widehat{\mathbf{v}}_{I_k}, \quad {}^{I_k} \widehat{\mathbf{g}}_k = {}^G \widehat{\mathbf{R}}_{I_k}^T \widehat{\mathbf{g}} \\ {}^{I_k} \check{\mathbf{R}}_{I_{j-1}} &= {}^{I_k} \check{\mathbf{R}}_{I_j} \text{Exp} \left( \left( \widehat{\mathbf{b}}_{\omega_k} - \boldsymbol{\omega}_{m_{i-1}} \right) \Delta t \right), \quad \text{s.f. } {}^{I_k} \mathbf{R}_{I_m} = \mathbf{I}. \end{aligned}$$

s.f. 表示"starting from"。注意这一公式是在IMU坐标系下。

## 运动补偿

定义我们反向传播得到的点的相对位姿是  ${}^{I_k}\check{\mathbf{T}}_{I_j} = ({}^{I_k}\check{\mathbf{R}}_{I_j}, {}^{I_k}\check{\mathbf{p}}_{I_j})$ ，我们把该点  ${}^{L_j}\mathbf{p}_{f_j}$  投影到扫描结束时间  $t_k$ ：

$${}^{L_k}\mathbf{p}_{f_j} = {}^I\mathbf{T}_L^{-1} {}^{I_k}\check{\mathbf{T}}_{I_j} {}^I\mathbf{T}_L {}^{L_j}\mathbf{p}_{f_j}. \quad (2)$$

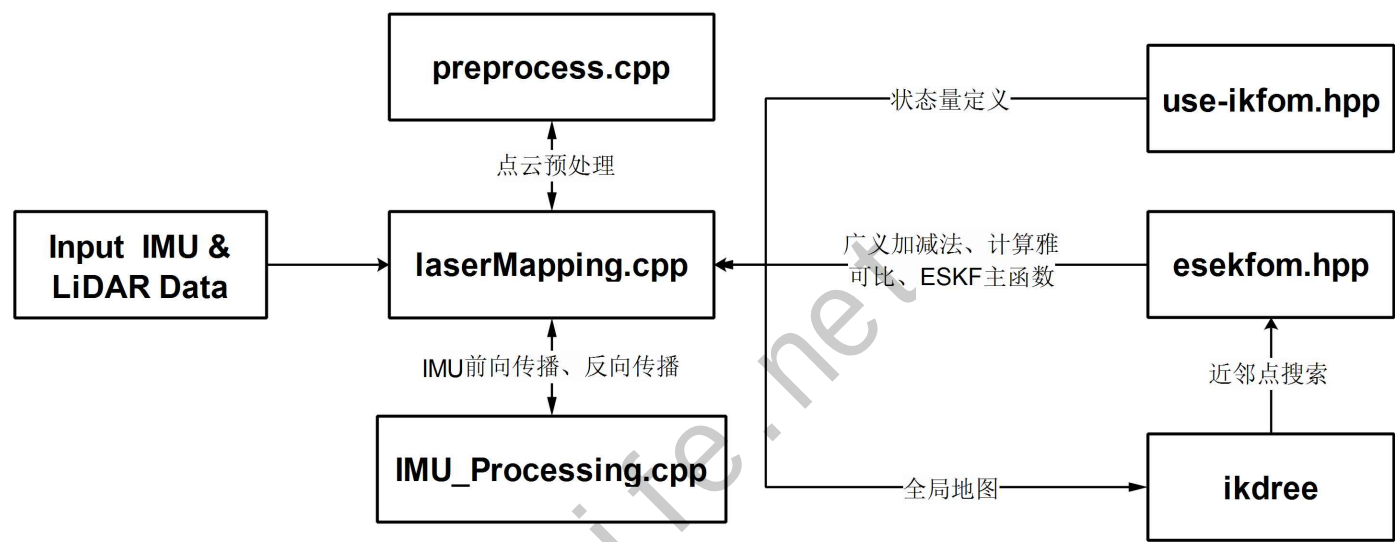
- ${}^I\mathbf{T}_L$  表示雷达坐标系转到IMU坐标系的外参矩阵
- 先把L坐标系的特征点转到I坐标系，然后乘上反向传播的补偿矩阵，再转回L坐标系

cvlife.net

# 9.代码讲解：IMU和LIDAR数据预处理

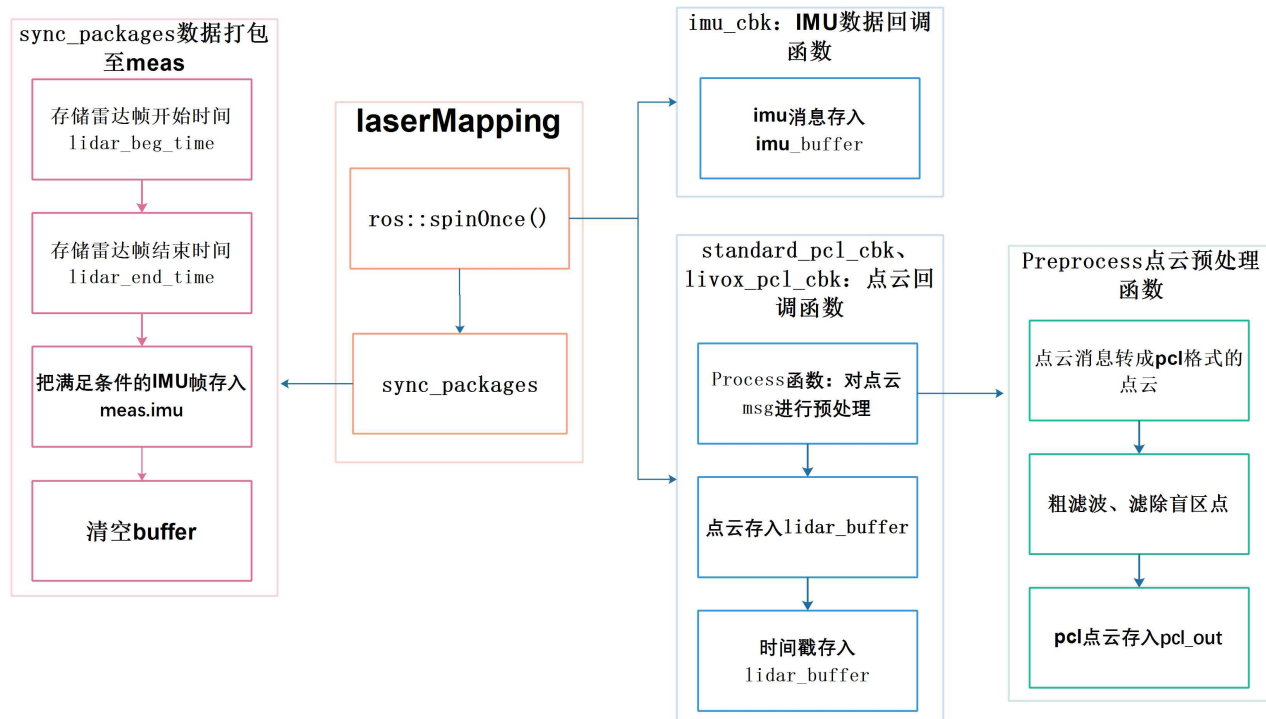
## 代码总体框架

S-FAST-LIO代码结构一图流



- `laserMapping.cpp`: 主函数，负责统筹所有进程，接收话题，建图，可视化
- `IMU_Processing.cpp`: IMU前向传播、反向传播，把传播后的状态量、协方差矩阵、点云返回`laserMapping`
- `preprocess.cpp`: 对点云进行预处理，格式转换
- `use-ikfom.hpp`: 状态量的定义，生成前向传播的状态转移矩阵
- `esekfom.hpp`: 广义加减法定义，前向传播函数，计算残差以及雅可比，ESKF主函数

## 数据预处理和打包



cvlife.net

# 10.代码讲解：IMU初始化与状态传播

## use-ikfom.hpp

定义状态量 $\mathbf{x}$ ，输入 $\mathbf{u}$ 和噪声 $\mathbf{Q}$

计算 $\mathbf{f}$ 、 $\mathbf{f_x}$ 、 $\mathbf{f_w}$ 矩阵

$$\mathbf{f}(\mathbf{x}_i, \mathbf{u}_i, \mathbf{w}_i) = \begin{bmatrix} \omega_{m_i} - \mathbf{b}_{\omega_i} - \mathbf{n}_{\omega_i} \\ {}^G\mathbf{v}_{I_i} \\ {}^G\mathbf{R}_{I_i}(\mathbf{a}_{m_i} - \mathbf{b}_{\mathbf{a}_i} - \mathbf{n}_{\mathbf{a}_i}) + {}^G\mathbf{g}_i \\ \mathbf{n}_{\mathbf{b}\omega_i} \\ \mathbf{n}_{\mathbf{b}\mathbf{a}_i} \\ \mathbf{0}_{3 \times 1} \end{bmatrix}$$

$$\mathbf{F_x} = \begin{bmatrix} \text{Exp}(-\hat{\omega}_i \Delta t) & \mathbf{0} & \mathbf{0} & -\mathbf{A}(\hat{\omega}_i \Delta t)^T \Delta t & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{I} & \mathbf{I} \Delta t & \mathbf{0} & \mathbf{0} & \mathbf{0} \\ -{}^G\hat{\mathbf{R}}_{I_i}[\hat{\mathbf{a}}_i]_{\wedge} \Delta t & \mathbf{0} & \mathbf{I} & \mathbf{0} & -{}^G\hat{\mathbf{R}}_{I_i} \Delta t & \mathbf{I} \Delta t \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{I} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{I} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{I} \end{bmatrix}$$

$$\mathbf{F_w} = \begin{bmatrix} -\mathbf{A}(\hat{\omega}_i \Delta t)^T \Delta t & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & -{}^G\hat{\mathbf{R}}_{I_i} \Delta t & \mathbf{0} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{I} \Delta t & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{I} \Delta t & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} \end{bmatrix}$$

## laserMapping

ros::spinOnce()

sync\_packages

Process()

## IMU\_Processing.hpp

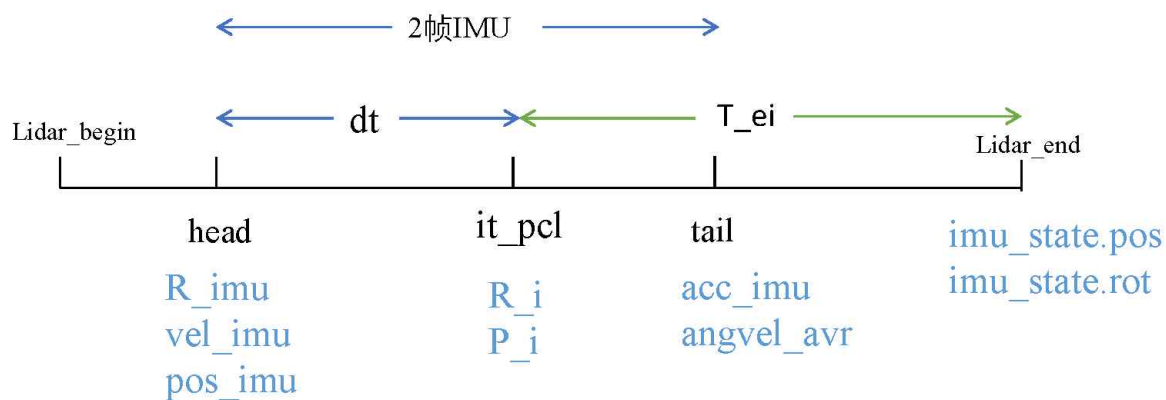
IMU\_init: 初始化重力向量和 $\mathbf{bias}$ , 初始化状态量 $\mathbf{x}$   
初始化协方差 $\mathbf{P}$

UndistortPcl函数

对于每一帧IMU, 计算时间间隔 $\Delta t$ , 协方差矩阵 $\mathbf{Q}$

前向传播计算状态量 $\mathbf{x}$   
(predict函数)

反向传播运动补偿, 补偿后的点云为pcl\_out



点it\_pcl所在时刻的旋转：

**M3D**  $R_i (R_{imu} * \text{Exp}(\text{angvel\_avr}, dt))$

点所在时刻的位置(Lidar坐标系)：

**V3D**  $P_i (it\_pcl \rightarrow x, it\_pcl \rightarrow y, it\_pcl \rightarrow z)$

从点所在的世界位置-雷达末尾世界位置：

**V3D**  $T_{ei} (pos\_imu + vel\_imu * dt + 0.5 * acc\_imu * dt * dt - imu\_state.pos)$

运动补偿： **V3D**  $P\_compensate = imu\_state.offset\_R\_L\_l.matrix().transpose() * (imu\_state.rot.matrix().transpose() * (R_i * (imu\_state.offset\_R\_L\_l.matrix() * P_i + imu\_state.offset\_T\_L\_l) + T_{ei}) - imu\_state.offset\_T\_L\_l)$

$$imustate.rot \times ({}^l R_L \times P_{com} + {}^l T_L) + imustate.pos = R_i ({}^l R_L \times P_i + {}^l T_L) + pos_{pcl}$$

$$T_{ei} = pos_{pcl} - imustate.pos$$

补偿后的点在世界坐标系下的位姿 = 补偿前的点在世界坐标系下的位姿

# 卡尔曼滤波 知识回顾

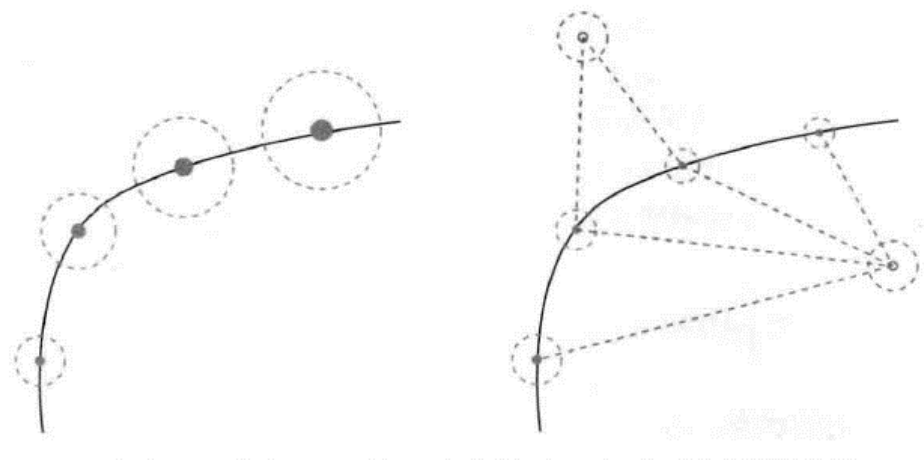
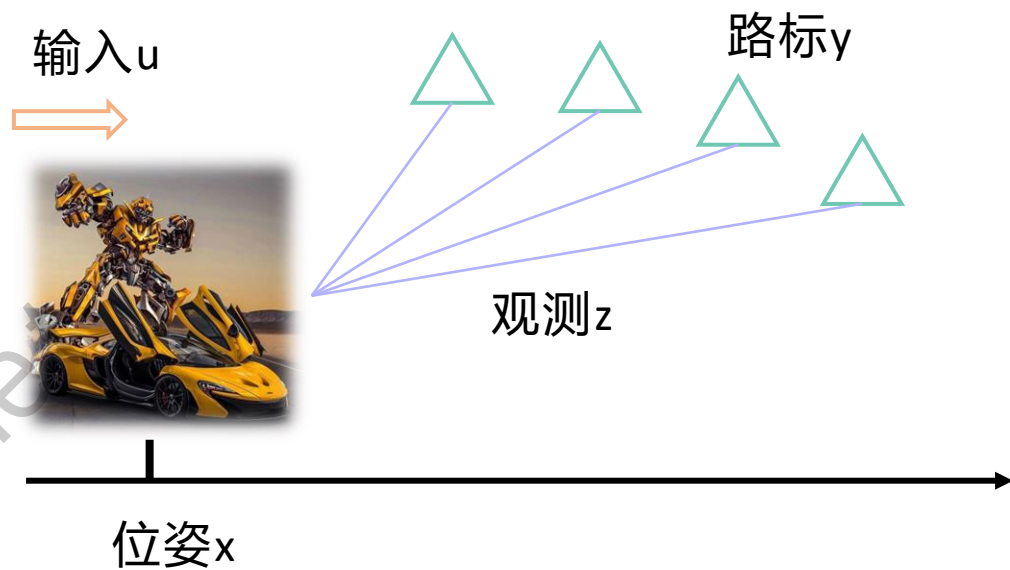


# ● 状态估计问题

## ➤ SLAM运动方程与观测方程

$$\begin{cases} x_k = f(x_{k-1}, u_k) + w_k \\ z_{k,j} = h(y_j, x_k) + v_{k,j} \end{cases}, k = 1, \dots, N, j = 1, \dots, M.$$

- 把位姿 $x$ 看作是服从某种概率分布的随机变量
- 运动方程受噪声影响，误差逐渐累积
- 通过观测数据减小不确定性
- 两种状态估计方式：**批量式**（非线性优化）和**增量式**（滤波）



# 最大后验估计

➤ 令:  $x = \{x_1, \dots, x_N\}$ ,  $y = \{y_1, \dots, y_M\}$ .

➤ SLAM问题即是求:  $P(x, y | z, u)$ .

$$P(x, y | z, u) = \frac{P(z, u | x, y) P(x, y)}{P(z, u)} \propto P(z, u | x, y) P(x, y)$$

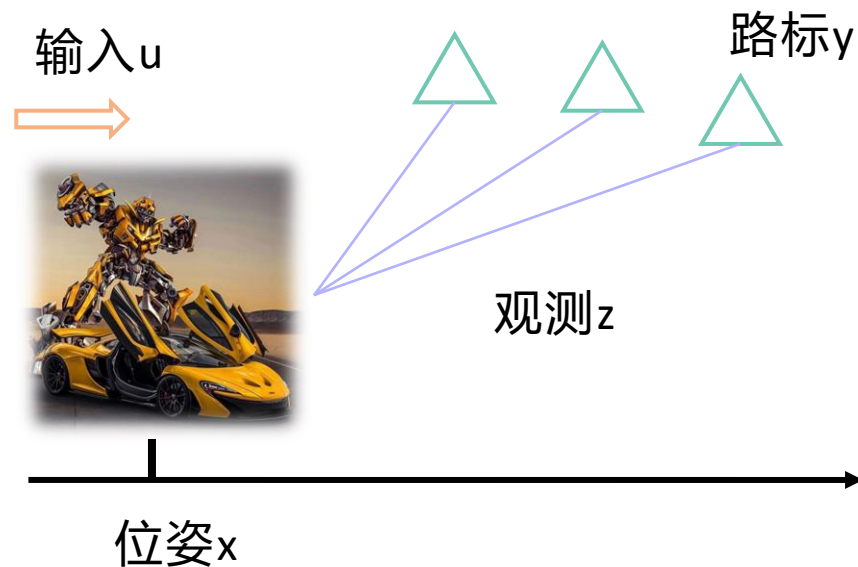
似然      先验

$$(x, y)_{\text{MAP}}^* = \arg \max P(x, y | z, u) = \arg \max P(z, u | x, y) P(x, y).$$

$$(x, y)_{\text{MLE}}^* = \arg \max P(z, u | x, y)$$

- 直接求后验概率分布是困难的, 我们转而求一个状态最优估计, 使得该状态下后验概率最大化
- 在什么样的状态下, 最可能产生现在拿到的观测数据**

$$\begin{cases} x_k = f(x_{k-1}, u_k) + w_k \\ z_{k,j} = h(y_j, x_k) + v_{k,j} \end{cases}, k = 1, \dots, N, j = 1, \dots, M.$$



# ● 最小二乘问题

$$\begin{cases} x_k = f(x_{k-1}, u_k) + w_k \\ z_{k,j} = h(y_j, x_k) + v_{k,j} \end{cases}, k = 1, \dots, N, j = 1, \dots, M.$$

➤ 对联合分布进行因式分解:  $P(z, u | x, y) = \prod_k P(u_k | x_{k-1}, x_k) \prod_{k,j} P(z_{k,j} | x_k, y_j)$

- 假设高斯分布, 某一次观测:

$$P(z_{j,k} | x_k, y_j) = N(h(y_j, x_k), Q_{k,j})$$

- 任意高斯分布:  $x \sim N(\mu, \Sigma)$

$$P(x) = \frac{1}{\sqrt{(2\pi)^N \det(\Sigma)}} \exp\left(-\frac{1}{2}(x - \mu)^T \Sigma^{-1}(x - \mu)\right)$$

$$-\ln(P(x)) = \frac{1}{2} \ln((2\pi)^N \det(\Sigma)) + \frac{1}{2}(x - \mu)^T \Sigma^{-1}(x - \mu)$$

- 等价于一个二次型:

$$\arg \max \mathcal{N}(h(y_j, x_k), Q_{k,j})$$

$$= \arg \min ((z_{k,j} - h(x_k, y_j))^T Q_{k,j}^{-1} (z_{k,j} - h(x_k, y_j)))$$

与x无关

- 定义输入、观测数据与模型之间的误差:

$$e_{u,k} = x_k - f(x_{k-1}, u_k)$$

$$e_{z,j,k} = z_{k,j} - h(x_k, y_j)$$

$$\min J(x, y) = \sum_k e_{u,k}^T R_k^{-1} e_{u,k} + \sum_k \sum_i e_{z,k,j}^T Q_{k,j}^{-1} e_{z,k,j}$$

✓ 可通过非线性优化方式求解

# ● 卡尔曼滤波(KF)

➤ 令:  $x_k \stackrel{\text{def}}{=} \{x_k, y_1, \dots, y_m\}$ .

➤ 马尔可夫性假设: k时刻状态只与k-1时刻状态有关

$$P(x_k | z_k, u_k, x_{k-1}) \propto P(z_k | x_k) P(x_k | u_k, x_{k-1})$$

似然

先验

• 假设系统为线性高斯系统

$$\begin{cases} x_k = A_k x_{k-1} + u_k + w_k \\ z_k = C_k x_k + v_k \end{cases} \cdot w_k \sim N(\mathbf{0}, R), v_k \sim N(\mathbf{0}, Q).$$

✓ 预测 (先验概率分布):

$$\bar{x}_k = A_k \hat{x}_{k-1} + u_k, \bar{P}_k = A_k \hat{P}_{k-1} A_k^T + R$$

$$\begin{cases} x_k = f(x_{k-1}, u_k) + w_k \\ z_{k,j} = h(y_j, x_k) + v_{k,j} \end{cases}, k = 1, \dots, N, j = 1, \dots, M.$$

• 根据  $P(z_k | x_k) = N(C_k x_k, Q)$

$$N(\hat{x}_k, \hat{P}_k) = \eta N(C_k x_k, Q) \cdot N(\bar{x}_k, \bar{P}_k)$$

• 通过比较等式两边的系数:



✓ 更新 (后验概率分布):

$$K = \bar{P}_k C_k^T (C_k \bar{P}_k C_k^T + Q_k)^{-1}$$

$$\hat{x}_k = \bar{x}_k + K(z_k - C_k \bar{x}_k)$$

$$\hat{P}_k = (I - KC_k) \bar{P}_k$$

## ● 扩展卡尔曼滤波(EKF)

➤ 对于非线性系统，通过一阶泰勒展开：

$$x_k \approx f(\hat{x}_{k-1}, u_k) + \left. \frac{\partial f}{\partial x_{k-1}} \right|_{\hat{x}_{k-1}} (x_{k-1} - \hat{x}_{k-1}) + w_k$$

$$z_k \approx h(\bar{x}_k) + \left. \frac{\partial h}{\partial x_k} \right|_{\bar{x}_k} (x_k - \bar{x}_k) + n_k$$

• 记  $F = \left. \frac{\partial f}{\partial x_{k-1}} \right|_{\hat{x}_{k-1}}, H = \left. \frac{\partial h}{\partial x_k} \right|_{\bar{x}_k}$

✓ 更新（后验概率分布）：

$$K_k = \bar{P}_k H^T (H \bar{P}_k H^T + Q_k)^{-1}$$

$$\hat{x}_k = \bar{x}_k + K_k (z_k - h(\bar{x}_k)), \hat{P}_k = (I - K_k H) \bar{P}_k$$

$$\begin{cases} x_k = f(x_{k-1}, u_k) + w_k \\ z_{k,j} = h(y_j, x_k) + v_{k,j} \end{cases}, k = 1, \dots, N, j = 1, \dots, M.$$

- EKF做了线性化处理，在离工作点较远时线性化程度不够

## ● 迭代扩展卡尔曼滤波(iEKF)

$$\begin{cases} x_k = f(x_{k-1}, u_k) + w_k \\ z_{k,j} = h(y_j, x_k) + v_{k,j} \end{cases}, k = 1, \dots, N, j = 1, \dots, M.$$

➤ 在k推到k+1的过程中进行多次迭代，减小非线性的影响（误差）

• 与EKF相同的线性化：

$$h_k(x_k) = h_k(\hat{x}_{k|k-1}) + H_k(x_k - \hat{x}_{k|k-1}) + o(x_k - \hat{x}_{k|k-1})$$

$$f_k(x_k) = f_k(\hat{x}_{k|k-1}) + F_k(x_k - \hat{x}_{k|k-1}) + o(x_k - \hat{x}_{k|k-1})$$

□ 步骤1:  $\hat{x}_{k|k-1}^0 = \hat{x}_{k|k-1}$

□ 步骤2: 计算  $\hat{x}_{k|k-1}^i$  处的雅可比矩阵和增益矩阵，重复至满足收敛条件

$$H_k^i = \frac{\partial h_k(s)}{\partial s} \Big|_{s = \hat{x}_k^i} \quad K_k^i = P_{k|k-1} [H_k^i]^T (H_k^i P_{k|k-1} [H_k^i]^T + R)^{-1}$$
$$\hat{x}_k^{i+1} = \hat{x}_{k|k-1} + K_k^i (z_k - h_k(\hat{x}_k^i) - H_k^i (\hat{x}_{k|k-1} - \hat{x}_k^i))$$

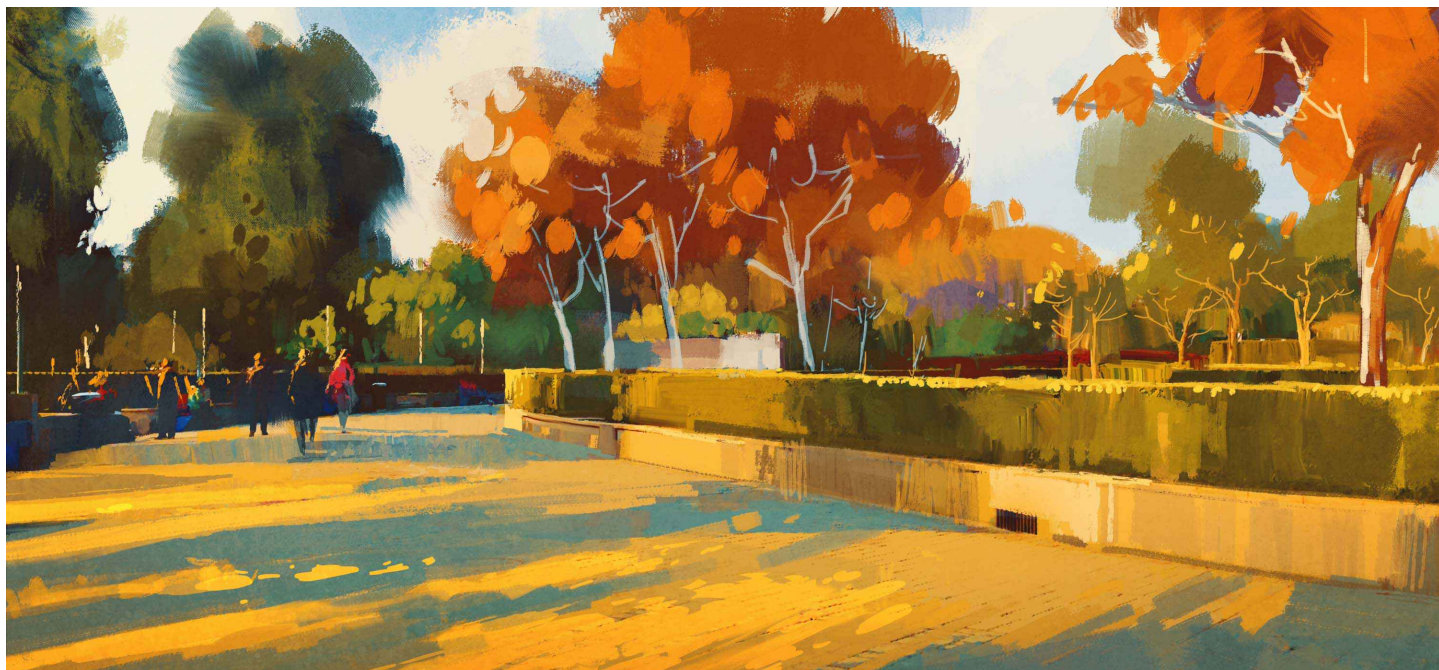
□ 步骤3: 更新x和协方差P

$$\hat{x}_{k|k-1} = \hat{x}_k^i, P_{k|k} = (I - K_k^i H_k^i) P_{k|k-1} (I - K_k^i H_k^i)^T + K_k^i R K_k^{iT}$$

✓ 减小非线性影响

✓ 与高斯-牛顿法等价

谢谢



## 12. 误差状态卡尔曼滤波器 (ESKF)

### 误差状态卡尔曼滤波器简介

传统EKF的运动方程和观测方程：

$$\begin{aligned}\mathbf{x}_k &= \mathbf{f}(\mathbf{x}_{k-1}, \mathbf{u}_k) + \mathbf{w}_k \\ \mathbf{z}_k &= \mathbf{h}(\mathbf{x}_k) + \mathbf{v}_k\end{aligned}\tag{1}$$

误差状态卡尔曼滤波器的运动方程和观测方程：

$$\begin{aligned}\tilde{\mathbf{x}}_k &= \tilde{\mathbf{f}}(\tilde{\mathbf{x}}_{k-1}, \mathbf{u}_k) + \tilde{\mathbf{w}}_k \\ \mathbf{z}_k &= \tilde{\mathbf{h}}(\tilde{\mathbf{x}}_k) + \tilde{\mathbf{v}}_k\end{aligned}\tag{2}$$

即上式把  $\mathbf{x}$  用  $\tilde{\mathbf{x}}$  代替。有了误差量的估计，再通过  $\mathbf{x} = \hat{\mathbf{x}} \oplus \tilde{\mathbf{x}}$  计算最优估计。这将带来以下好处：

- ESKF的状态变量可以采用最小化的参数表达，而传统KF需要用到四元数(4维)或者更高维的表达(旋转矩阵, 9维)
- ESKF总是在原点附近，离奇异点较远，并且也不会由于离工作点太远而导致线性化近似不够的问题
- ESKF的状态量为小量，其二阶变量相对来说可以忽略。
- 大多数雅可比矩阵在小量情况下变得非常简单，甚至可以用单位阵代替

## 残差的计算

特征点转换到全局坐标系G:

$${}^G\hat{\mathbf{p}}_{f_j}^\kappa = {}^G\hat{\mathbf{T}}_{I_k}^\kappa {}^I\mathbf{T}_L^{L_k} \mathbf{p}_{f_j}; j = 1, \dots, m \quad (3)$$

其中  $\hat{\mathbf{T}}_{I_k}^\kappa$  是待求的位姿变换。

定义  $\mathbf{u}_j$  为平面或边的法向量,  ${}^G\mathbf{q}_j$  表示平面或直线上的一点, 则残差:

$$\mathbf{z}_j^\kappa = \mathbf{G}_j \left( {}^G\hat{\mathbf{p}}_{f_j}^\kappa - {}^G\mathbf{q}_j \right) \quad (4)$$

## ESKF观测方程

测量噪声来源于雷达的测距噪声, 去除这些噪声可以得到真实的点的位置:

$${}^{L_j}\mathbf{p}_{f_j}^{\text{gt}} = {}^{L_j}\mathbf{p}_{f_j} - {}^{L_j}\mathbf{n}_{f_j} \quad (5)$$

理论上, 如果我们使用状态量的真值以及真实的雷达点, 其残差应当为零:

$$\mathbf{0} = \mathbf{h}_j(\mathbf{x}_k, {}^{L_j}\mathbf{n}_{f_j}) = \mathbf{G}_j \left( {}^G\mathbf{T}_{I_k}^{I_k} \tilde{\mathbf{T}}_{I_j}^{I_k} {}^I\mathbf{T}_L \left( {}^{L_j}\mathbf{p}_{f_j} - {}^{L_j}\mathbf{n}_{f_j} \right) - {}^G\mathbf{q}_j \right) \quad (6)$$

一阶近似, 在  $\tilde{\mathbf{x}}_k^\kappa$  附近展开 (即在  $\tilde{\mathbf{x}}_k^\kappa = \mathbf{0}$  处, 因为  $\tilde{\mathbf{x}}_k^\kappa = \mathbf{x}_k \boxminus \hat{\mathbf{x}}_k^\kappa$ ):

$$\begin{aligned} \mathbf{0} &= \mathbf{h}_j(\mathbf{x}_k, {}^{L_j}\mathbf{n}_{f_j}) = \mathbf{h}_j(\hat{\mathbf{x}}_k^\kappa \boxplus \tilde{\mathbf{x}}_k^\kappa, {}^{L_j}\mathbf{n}_{f_j}) \\ &\simeq \mathbf{h}_j(\hat{\mathbf{x}}_k^\kappa, \mathbf{0}) + \mathbf{H}_j^\kappa \tilde{\mathbf{x}}_k^\kappa + \mathbf{v}_j \\ &= \mathbf{z}_j^\kappa + \mathbf{H}_j^\kappa \tilde{\mathbf{x}}_k^\kappa + \mathbf{v}_j \end{aligned} \quad (7)$$

上式即为ESKF的观测方程。

## 观测方程雅可比推导

$\mathbf{H}_j^\kappa$  是  $h(x, n)$  关于  $\tilde{\mathbf{x}}_k^\kappa$  的雅可比矩阵, 论文中并未给出详细的推导, 这里我们推导一下这个雅可比矩阵, 它与代码中的形式是一致的。

利用链式法则:

$$\mathbf{H} = \frac{\partial \mathbf{h}}{\partial \mathbf{p}} \frac{\partial \mathbf{p}}{\partial \tilde{\mathbf{x}}} \quad (8)$$

其中  $p$  表示转换到{G}系下的特征点。设在{L}坐标系下的特征点为  ${}^L p$ ，则

$$\mathbf{p} = {}^G \mathbf{R}_I ({}^I \mathbf{R}_L {}^L \mathbf{p} + {}^I \mathbf{T}_L) + {}^G \mathbf{T}_I \quad (9)$$

残差定义为：

$$\mathbf{h} = (\mathbf{p} - \mathbf{p}_a) \cdot \vec{\mathbf{n}} \quad (10)$$

其中  $p_a$  为该平面上的一点。则

$$\frac{\partial \mathbf{h}}{\partial \mathbf{p}} = \vec{\mathbf{n}} \quad (11)$$

接下来求解  $\frac{\partial \mathbf{p}}{\partial \delta \tilde{\mathbf{x}}}$  (在  $\tilde{\mathbf{x}} = 0$  处)。实际上该导数只与旋转和平移有关，也就是该矩阵只需要前6列，其他行都为零。即：

$$\frac{\partial \mathbf{p}}{\partial \tilde{\mathbf{x}}} = [-{}^G \hat{\mathbf{R}}_I ({}^I \mathbf{R}_L {}^L \mathbf{p} + {}^I \mathbf{T}_L)^\wedge, I_{3 \times 3}, 0, 0, 0, 0] \quad (12)$$

其中第一项利用了李代数求导的右扰动模型。将以上2式相乘即可得到：

$$\mathbf{H} = [-\vec{\mathbf{n}} {}^G \hat{\mathbf{R}}_I ({}^I \mathbf{R}_L {}^L \mathbf{p} + {}^I \mathbf{T}_L)^\wedge, \vec{\mathbf{n}}, 0, 0, 0, 0] \quad (13)$$

## ESKF迭代更新

ESKF的**运动方程**实际上即为公式（5）的误差量传递方程。考虑上噪声之后，它的理论值应当为零：

$$\mathbf{0} = \tilde{\mathbf{x}}_k = \Sigma (\mathbf{F}_{\tilde{\mathbf{x}}} \tilde{\mathbf{x}}_i + \mathbf{F}_{\mathbf{w}} \mathbf{w}_i) \sim N(\mathbf{0}, \hat{\mathbf{P}}_k) \quad (14)$$

在IEKF中，迭代过程不会更新协方差矩阵P。而文中用了如下公式，使得我们在迭代过程中也能够更新协方差矩阵：

$$\tilde{\mathbf{x}}_k = \mathbf{x}_k \boxminus \hat{\mathbf{x}}_k = (\hat{\mathbf{x}}_k^\kappa \boxplus \tilde{\mathbf{x}}_k^\kappa) \boxminus \hat{\mathbf{x}}_k = \hat{\mathbf{x}}_k^\kappa \boxminus \hat{\mathbf{x}}_k + \mathbf{J}^\kappa \tilde{\mathbf{x}}_k^\kappa \sim N(\mathbf{0}, \hat{\mathbf{P}}_k) \quad (15)$$

通过上式可推得：

$$\tilde{\mathbf{x}}_k^\kappa \sim N(\mathbf{0}, (\mathbf{J}^\kappa)^{-1} \hat{\mathbf{P}}_k (\mathbf{J}^\kappa)^{-T}) \quad (16)$$

最大后验估计（MAP）：

$$\min_{\hat{\mathbf{x}}_k^\kappa} \left( \|\mathbf{x}_k \ominus \hat{\mathbf{x}}_k\|_{\hat{\mathbf{P}}_k^{-1}}^2 + \sum_{j=1}^m \|\mathbf{z}_j^\kappa + \mathbf{H}_j^\kappa \hat{\mathbf{x}}_k^\kappa\|_{\mathbf{R}_j^{-1}}^2 \right) \quad (17)$$

卡尔曼增益：

$$\mathbf{K} = \mathbf{P}\mathbf{H}^T (\mathbf{H}\mathbf{P}\mathbf{H}^T + \mathbf{R})^{-1} \quad (18)$$

$$\hat{\mathbf{x}}_k^{\kappa+1} = \hat{\mathbf{x}}_k^\kappa \ominus \left( -\mathbf{K}\mathbf{z}_k^\kappa - (\mathbf{I} - \mathbf{K}\mathbf{H}) (\mathbf{J}^\kappa)^{-1} (\hat{\mathbf{x}}_k^\kappa \ominus \hat{\mathbf{x}}_k) \right) \quad (19)$$

## 高斯牛顿法回顾

考虑一个最小二乘问题：

$$\min_x F(x) = \frac{1}{2} \|f(x)\|_2^2 \quad (20)$$

一阶近似：

$$f(x + \Delta x) \approx f(x) + J(x)^T \Delta x \quad (21)$$

该问题可转换为寻找增量  $\Delta x$ ，使  $\|f(x + \Delta x)\|^2$  最小：

$$\Delta x^* = \arg \min_{\Delta x} \frac{1}{2} \left\| f(x) + J(x)^T \Delta x \right\|^2 \quad (22)$$

$$\frac{1}{2} \|f(x) + J(x)^T \Delta x\|^2 = \frac{1}{2} (f(x) + J(x)^T \Delta x)^T (f(x) + J(x)^T \Delta x) = \frac{1}{2} (\|f(x)\|_2^2 + 2f(x)^T J(x)^T \Delta x + \Delta x^T J(x) J(x)^T \Delta x). \quad (23)$$

上式对于  $\Delta x$  求导，并令其为零：

$$J(x) f(x) + J(x) J^T(x) \Delta x = 0 \quad (24)$$

即：

$$H \Delta x = g \quad (25)$$

高斯牛顿步骤：

1. 给定初值  $x_0$

2. 求当前雅可比J和误差f
3. 求解  $H\Delta x = g$
4. 若  $\Delta x$  足够小，则停止，否则  $x_{k+1} = x_k + \Delta x_k$ ，返回2继续迭代

## ESKF迭代更新与高斯牛顿的等价性

省略掉下标k，令  $X = \hat{x}^\kappa \triangleq \hat{x}$ ，最大后验估计可以表示为：

$$E = \min_{\tilde{x}^\kappa} \frac{1}{2} \begin{pmatrix} z^\kappa + H\tilde{x}^\kappa \\ X + J^\kappa \tilde{x}^\kappa \end{pmatrix}^T \begin{pmatrix} R & \\ & \hat{P} \end{pmatrix}^{-1} \begin{pmatrix} z^\kappa + H\tilde{x}^\kappa \\ X + J^\kappa \tilde{x}^\kappa \end{pmatrix} = \min_{\tilde{x}^\kappa} \frac{1}{2} M^T M \quad (26)$$

其中

$$M = S \begin{pmatrix} z^\kappa + H\tilde{x}^\kappa \\ X + J^\kappa \tilde{x}^\kappa \end{pmatrix}, S^T S = \begin{pmatrix} R & \\ & \hat{P} \end{pmatrix}^{-1} \quad (27)$$

显然M关于  $\tilde{x}^\kappa$  的雅可比为：

$$\mathcal{J} = (H^T \quad J^{\kappa T}) S^T \quad (28)$$

根据高斯牛顿法，每次迭代的增量：

$$\Delta \tilde{x}^\kappa = -(\mathcal{J} \mathcal{J}^T)^{-1} \mathcal{J} M \quad (29)$$

分别推导：

$$\begin{aligned} (\mathcal{J} \mathcal{J}^T)^{-1} &= \left[ (H^T \quad J^{\kappa T}) \begin{pmatrix} R^{-1} & \\ & \hat{P}^{-1} \end{pmatrix} \begin{pmatrix} H \\ J^\kappa \end{pmatrix} \right]^{-1} \\ &= (J^{\kappa T} \hat{P}^{-1} J^\kappa + H^T R^{-1} H)^{-1} \\ &\stackrel{SMW}{=} P - PH^T (R + HPH^T)^{-1} HP \\ &= (I - KH)P \\ &= KRH^{-T} \end{aligned} \quad (30)$$

其中  $P = J^{\kappa-1} \hat{P} J^{\kappa-T}$ ， $K = PH^T (HPH^T + R)^{-1}$ 。第三行利用了**SMW公式**：

$$(A^{-1} + BD^{-1}C)^{-1} = A - AB(D + CAB)^{-1}CA \quad (31)$$

最后一行推导如下：

$$\begin{aligned}
K &= PH^T (HPH^T + R)^{-1} \\
K(HPH^T + R) &= PH^T \\
KR - (I - KH)PH^T &= 0 \\
(I - KH)P &= KRH^{-T}
\end{aligned} \tag{32}$$

后面部分：

$$\begin{aligned}
-\mathcal{J}M &= - \begin{pmatrix} H^T & J^{\kappa T} \end{pmatrix} \begin{pmatrix} R^{-1} & \\ & \hat{P}^{-1} \end{pmatrix} \begin{pmatrix} z^\kappa + H\tilde{x}^\kappa \\ X + J^\kappa \tilde{x}^\kappa \end{pmatrix} \\
&= -H^T R^{-1}(z^\kappa + H\tilde{x}^\kappa) - J^{\kappa T} \hat{P}^{-1}(X + J^\kappa \tilde{x}^\kappa)
\end{aligned} \tag{33}$$

两式相乘：

$$\begin{aligned}
\Delta \tilde{x}^\kappa &= KRH^{-T}[-H^T R^{-1}(z^\kappa + H\tilde{x}^\kappa)] - (I - KH)P(J^{\kappa T} \hat{P}^{-1}(X + J^\kappa \tilde{x}^\kappa)) \\
&= -Kz^\kappa - KH\tilde{x}^\kappa - (I - KH)J^{\kappa-1} \hat{P} J^{\kappa-T} (J^{\kappa T} \hat{P}^{-1}(X + J^\kappa \tilde{x}^\kappa)) \\
&= -Kz^\kappa - KH\tilde{x}^\kappa - (I - KH)J^{\kappa-1}(X + J^\kappa \tilde{x}^\kappa) \\
&= -Kz^\kappa - \tilde{x}^\kappa - (I - KH)J^{\kappa-1}X
\end{aligned} \tag{34}$$

即：

$$\tilde{x}^{\kappa+1} = \tilde{x}^\kappa + \Delta \tilde{x}^\kappa = -Kz^\kappa - (I - KH)J^{\kappa-1}(\hat{\mathbf{x}}^\kappa \boxminus \hat{\mathbf{x}}) \tag{35}$$

## 新卡尔曼增益公式推导

最后，文中提到了一个工程问题，在计算卡尔曼增益时，矩阵  $\mathbf{H}\mathbf{P}\mathbf{H}^T + \mathbf{R}$  的维度是很大的（ $m \times m$ ，因为  $H_{m \times 18}, P_{18 \times 18}, R_{m \times m}$ ），因此文中给出了新的卡尔曼增益计算方式，它和原增益是等效的：

$$\mathbf{K} = (\mathbf{H}^T \mathbf{R}^{-1} \mathbf{H} + \mathbf{P}^{-1})^{-1} \mathbf{H}^T \mathbf{R}^{-1} \tag{36}$$

根据SMW公式：

$$(\mathbf{P}^{-1} + \mathbf{H}^T \mathbf{R}^{-1} \mathbf{H})^{-1} = \mathbf{P} - \mathbf{P} \mathbf{H}^T (\mathbf{H} \mathbf{P} \mathbf{H}^T + \mathbf{R})^{-1} \mathbf{H} \mathbf{P} \tag{37}$$

代入新的K：

$$\mathbf{K} = (\mathbf{H}^T \mathbf{R}^{-1} \mathbf{H} + \mathbf{P}^{-1})^{-1} \mathbf{H}^T \mathbf{R}^{-1} = \mathbf{P} \mathbf{H}^T \mathbf{R}^{-1} - \mathbf{P} \mathbf{H}^T (\mathbf{H} \mathbf{P} \mathbf{H}^T + \mathbf{R})^{-1} \mathbf{H} \mathbf{P} \mathbf{H}^T \mathbf{R}^{-1} \tag{38}$$

由于  $H^T P H^T R^{-1} = (H^T P H^T + R)R^{-1} - I$ , 则:

$$\begin{aligned} K &= P H^T \mathbf{R}^{-1} - P H^T \mathbf{R}^{-1} + P H^T (\mathbf{H} P \mathbf{R}^T + \mathbf{R})^{-1} \\ &= P H^T (\mathbf{H} P \mathbf{R}^T + \mathbf{R})^{-1}. \end{aligned} \tag{39}$$

和原卡尔曼增益等价。

cvlife.net

# 13.地图更新与实验结果分析

## 地图更新

根据最优状态量  $\bar{\mathbf{x}}_k$  ,  ${}^G\overline{\mathbf{T}}_{I_k} = ({}^G\overline{\mathbf{R}}_{I_k}, {}^G\overline{\mathbf{p}}_{I_k})$  把特征点全部投影到全局坐标系:

$${}^G\overline{\mathbf{p}}_{f_j} = {}^G\overline{\mathbf{T}}_{I_k} {}^I\mathbf{T}_L {}^{L_k}\mathbf{p}_{f_j}; j = 1, \cdots, m. \tag{1}$$

## IMU初始化

在开始时刻保持2s，利用收集到的数据初始化重力向量，IMU零偏等

## 实验

### 计算复杂度实验

将卡尔曼增益的计算替换为系统中的旧公式，并比较它们在相同系统和特征点数量下的计算时间

TABLE II  
THE RUNNING TIMES OF TWO KALMAN GAIN FORMULAS

| Feature Num.     | 307  | 717  | 998   | 1243 | 1453 | 1802 |
|------------------|------|------|-------|------|------|------|
| Old Formula (ms) | 7.1  | 23.4 | 109.3 | 251  | 1219 | 1621 |
| New Formula (ms) | 0.07 | 0.11 | 0.25  | 0.37 | 0.59 | 1.16 |

## 无人机飞行实验

- 无人机实验
- 70°FOV Livox Livox Avia LiDAR, DJI Manifold 2-C 1.8 GHz Intel i7-8550U CPU
- 最大50HZ的里程计输出+建图
- 漂移小于0.3\%（32米轨迹上的漂移为0.08米）

## 室内实验

- 室内手持快速旋转的场景( $>100^\circ/\text{s}$ )
- 与LOAM对比, 紧耦合vs松耦合

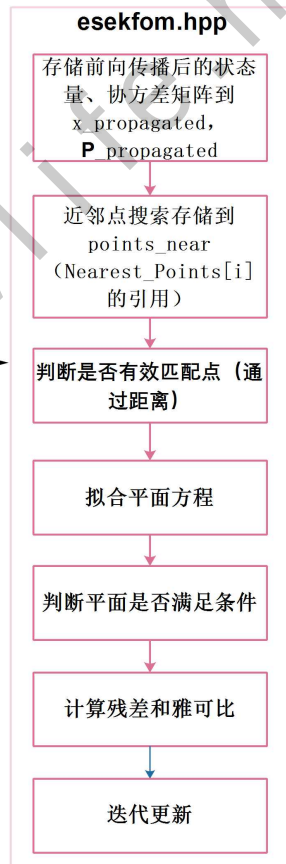
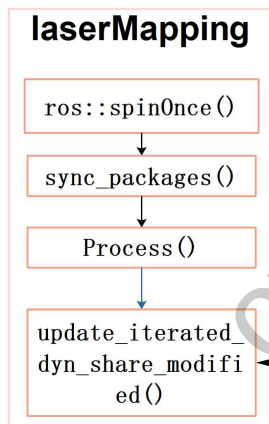
## 室外实验

- 室外手持, 香港大学主楼
- 漂移小于0.05\%(140米轨迹上的漂移为0.07米), 10HZ, 平均处理时间为25ms, 平均有效特征点为1497个
- 在LINS数据集上比较, FAST-LIO平均7.3ms, LINS平均34.5ms(差异主要在计算增益K的公式上)

cvlife.net



## 14.ESKF代码解析



$$\mathbf{H}_{all} = \begin{bmatrix} \mathbf{H}_{m \times 12} & \mathbf{0}_{m \times 12} \end{bmatrix}_{m \times 24}$$

$$\mathbf{K} = (\mathbf{H}^T \mathbf{R}^{-1} \mathbf{H} + \mathbf{P}^{-1})^{-1} \mathbf{H}^T \mathbf{R}^{-1}$$

$$\mathbf{H}^T \mathbf{R}^{-1} \mathbf{H} = \begin{bmatrix} \mathbf{H}^T \\ \mathbf{0} \end{bmatrix} \mathbf{R}^{-1} \begin{bmatrix} \mathbf{H} & \mathbf{0} \end{bmatrix} = \begin{bmatrix} \mathbf{H}^T \mathbf{H} / R & \mathbf{0} \\ \mathbf{0} & \mathbf{0} \end{bmatrix}$$

$$\mathbf{K}_{front} = (\mathbf{H}^T \mathbf{R}^{-1} \mathbf{H} + \mathbf{P}^{-1})^{-1}$$

$$\mathbf{K} = \mathbf{K}_{front} \mathbf{H}^T \mathbf{R}^{-1} = \mathbf{K}_{front} \begin{bmatrix} \mathbf{H}^T \\ \mathbf{0} \end{bmatrix} / R$$

$$= \begin{bmatrix} \mathbf{K}_{front\_12a} & \mathbf{K}_{front\_12b} \end{bmatrix} \begin{bmatrix} \mathbf{H}^T \\ \mathbf{0} \end{bmatrix} / R = \mathbf{K}_{front\_12a} \mathbf{H}^T / R$$

$$\mathbf{K} \mathbf{H} = \mathbf{K} \begin{bmatrix} \mathbf{H} & \mathbf{0} \end{bmatrix} = \begin{bmatrix} \mathbf{K} \mathbf{H} & \mathbf{0} \end{bmatrix}$$

$$Ax + By + Cz + D = 0 \Rightarrow \frac{A}{D}x + \frac{B}{D}y + \frac{C}{D}z + 1 = 0$$

$$\Rightarrow \frac{A}{Dn}x + \frac{B}{Dn}y + \frac{C}{Dn}z + \frac{1}{n} = 0 \quad (\text{参数归一化})$$

$$1 - 0.9 * \frac{d_{point-plane}}{d_{body\_norm}} > 0.9 \Rightarrow d_{point-plane} < \frac{1}{9} d_{body\_norm}$$

距离原点越远的值，距离容忍度越大

雅可比矩阵  $\mathbf{H}$

$$\mathbf{H} = [-\tilde{\mathbf{n}}^G \hat{\mathbf{R}}_I ({}^I \mathbf{R}_L {}^L \mathbf{p} + {}^I \mathbf{T}_L)^{\wedge}, \tilde{\mathbf{n}}, 0, 0, 0, 0]$$

# 15.ikdtree数据结构

## kdtree简介

[https://blog.csdn.net/qq\\_40692109/article/details/102653118](https://blog.csdn.net/qq_40692109/article/details/102653118)

<https://zhuanlan.zhihu.com/p/529926254>

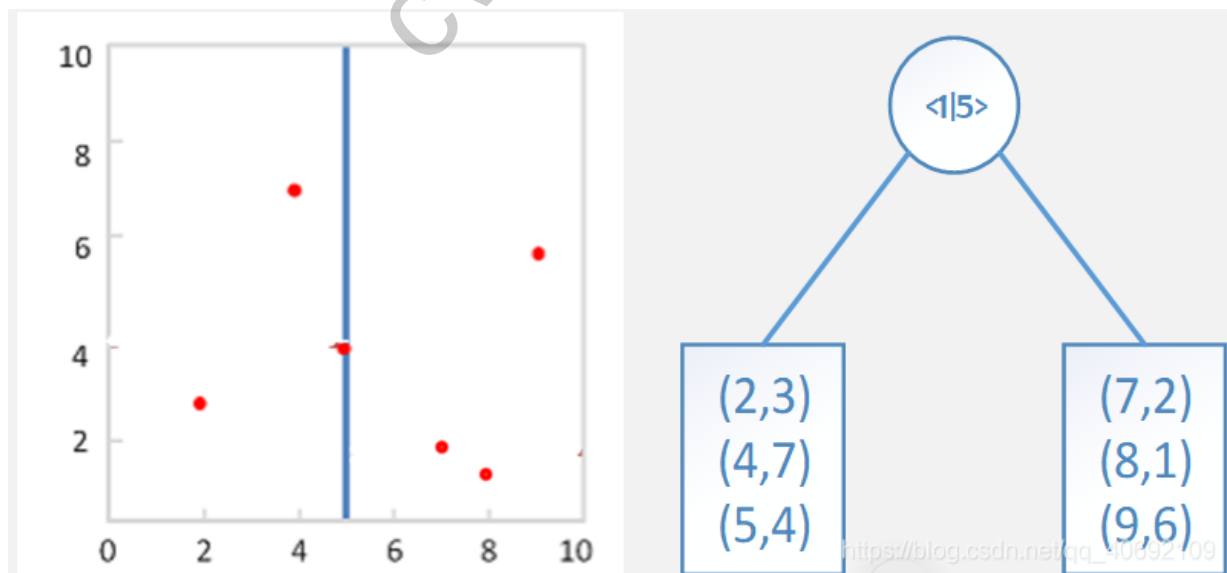
Kd-Tree (K-dimensional tree)，是一种高维索引树形数据结构，经常使用于在大规模的高维数据空间进行最近邻查找。

kd树结构类似于高维的二叉树，树中存储的是一些K维数据。

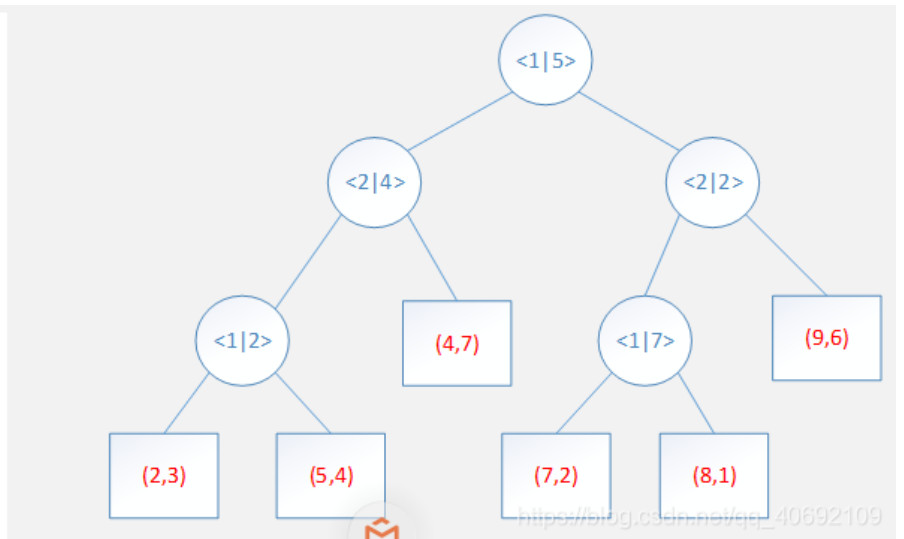
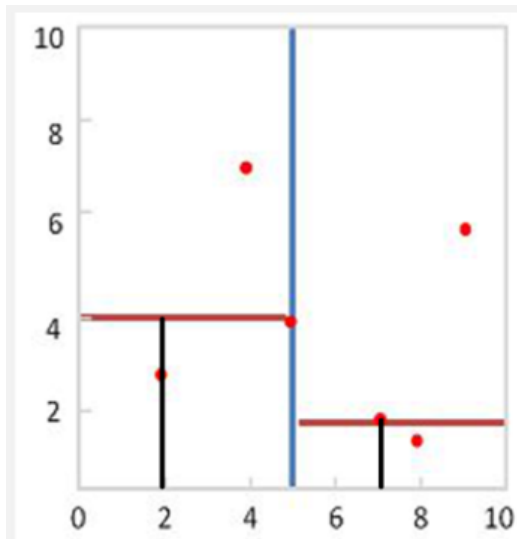
## kd-tree的构建步骤

1. 对数据在每个维度的方差进行计算，选取方差最大的维度作为划分维度 $k_v$ ;
2. 计算数据在 $k_v$ 维度的中位数，根据中位数将数据划分为2个子集，在 $k_v$ 维度上，小于等于中位数的数据放入左边子集，大于中位数的数据放入右边子集;
3. 对每个子集重复1,2步骤直至子集不能再划分为止

以二维数据为例，用 (2, 3) (4, 7) (5, 4) (7, 2) (8, 1) (9, 6)



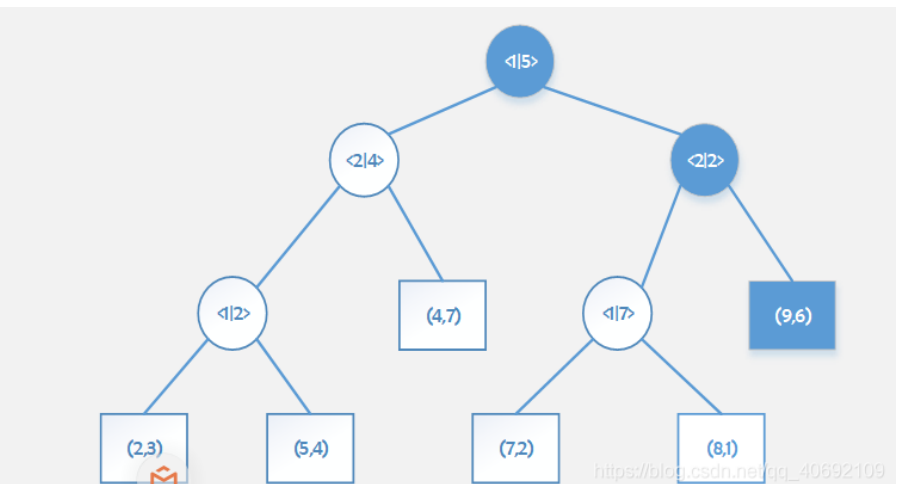
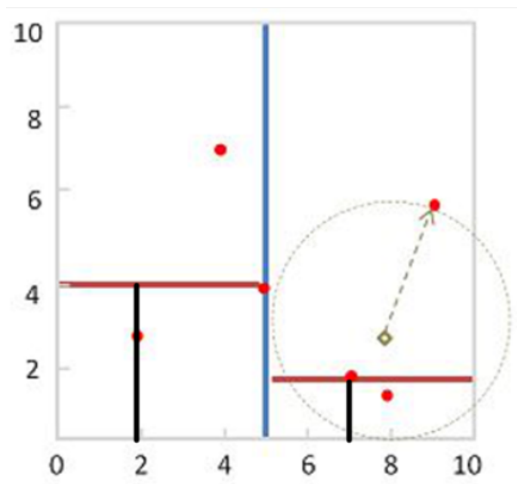
第一次构建：计算两个维度方差最大的维度（x维度），选取此维度中位（5）数进行划分。<1|5>节点表示在第1个维度上以5为划分值进行划分，小于等于5的划分到左子树，大于5的划分到右子树。

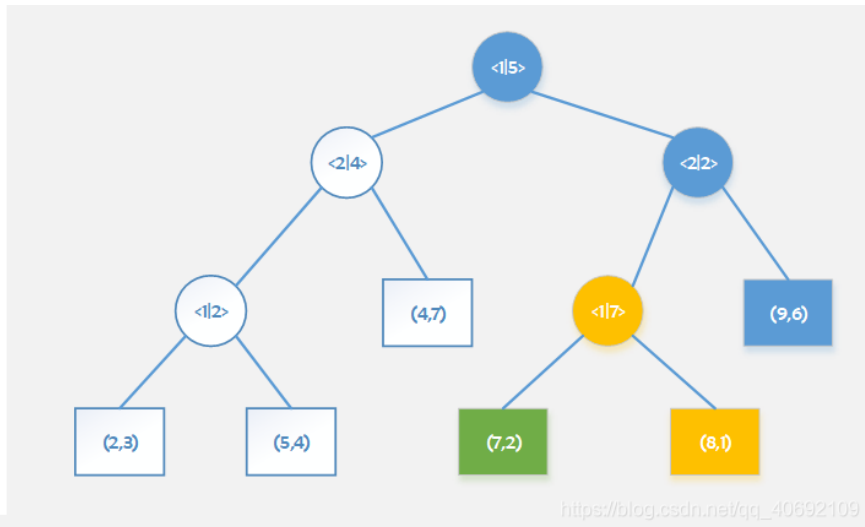
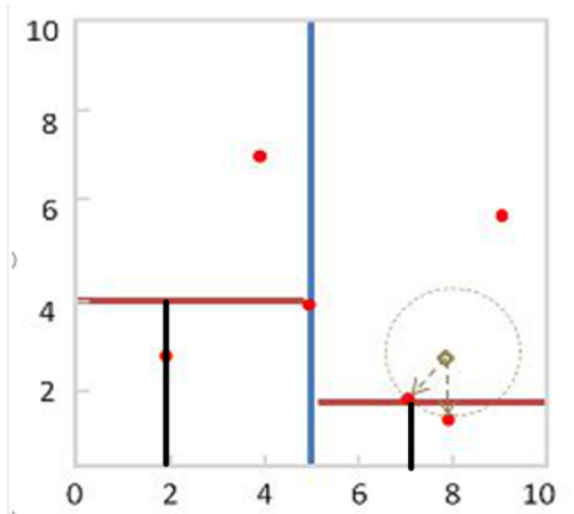


## kd-tree的近邻搜索

- 从根节点开始，对各个节点进行访问，直至访问至叶子结点，计算与叶子节点内数据的距离；
- 访问过程：在节点的维度与节点的划分大小进行比较；
- 因为每次访问时只在一个维度比较大小，所以此时得到的最近点并不一定为最小距离的点，未被访问的分支内话可能存在距离更小的点。还需要对未被访问的分支进行回溯；
- 进行回溯操作，判断其父节点下未被访问过的分支里是否还有更近的点，如果有进行同样操作，并更新距离；
- 回溯的判断过程是从下往上进行的，直到回溯到根结点时已经不存在与P更近的分支为止。

数据点集合：(2,3), (4,7), (5,4), (9,6), (8,1), (7,2)，需要搜索的点为 (8,3)。





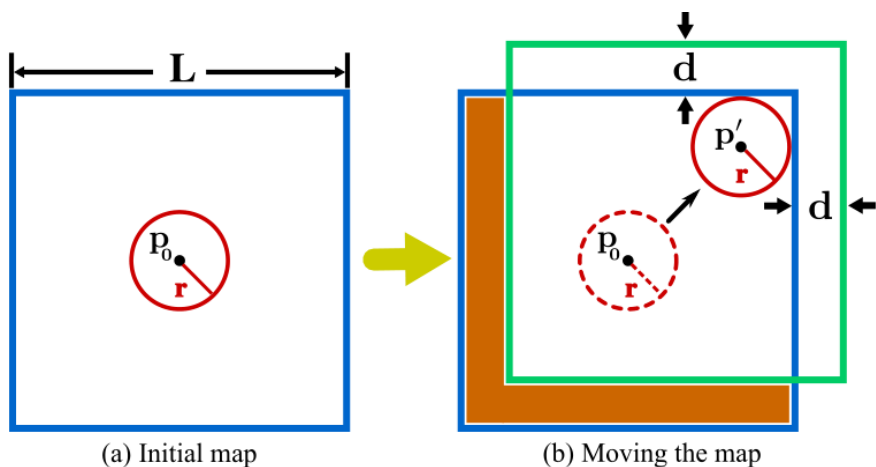
## kd-tree的特点

- 数据为多维数据，不同树节点根据不同维度进行数据划分
- 父节点将所有数据划分至左右子树，叶子节点包含所有数据
- 适用于动态程度不高的数据，插入删除不方便
- 不是动态增量式的结构，每次需要重新构建kdtree (A-LOAM)
- kdtree的深度会影响搜索速度，在建图过程中树的结构可能不平衡

## ikdtree

### 地图管理

- 地图由ikd-tree组成，仅保留当长度为L的大立方体的点云
- 当探测球体触碰到边界时，地图沿接触的边界方向移动
- 橙色部分的点云被删除，并新增点云



# 树的结构

---

## Data Structure: Tree node structure

---

```
1 Struct TreeNode:  
2   PointType point; 点的坐标 强度信息  
3   TreeNode * leftchild, * rightchild; 左右节点指针  
4   int axis; 分割轴  
5   int treesize, invalidnum; 树的大小 删除点数量  
6   bool deleted, treedeleted; 是否被删除(点, 子树)  
7   CuboidVertices range;  
8 end 子树上点的范围(外切长方体)
```

---

## 点的插入

- 首先将整个空间体素化，并明确新点落入哪个体素；
- 然后向ikd-Tree查询目标体素内是否已经有点以及有哪些点；
- 如果已经有点了，将已有点和新点一起排序，找出离体素中心最近的那个点，然后做判断：如果最近的点是已有点，意味着新点无必要再插入了，结束处理；如果最近的点是新点，则把已有点全部标记删除，并插入新点；
- 如果体素内尚不存在点，则直接插入新点。

---

### Algorithm 2: Point Insertion with On-tree Downsampling

---

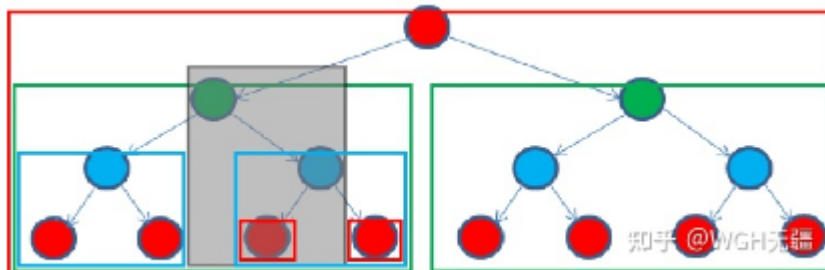
**Input:** Downsample Resolution  $l$ ,  
New Point to Insert  $p$ ,  
Switch of Parallely Re-building  $SW$

```
1 Algorithm Start  
2    $C_D \leftarrow \text{FindCube}(l, p)$   
3    $p_{\text{center}} \leftarrow \text{Center}(C_D)$ ;  
4    $V \leftarrow \text{BoxwiseSearch}(\text{RootNode}, C_D)$ ;  
5    $V.\text{push}(p)$ ;  
6    $p_{\text{nearest}} \leftarrow \text{FindNearest}(V, p_{\text{center}})$ ;  
7    $\text{BoxwiseDelete}(\text{RootNode}, C_D)$   
8    $\text{Insert}(\text{RootNode}, p_{\text{nearest}}, \text{NULL}, SW)$ ;  
9 Algorithm End
```

## 点的删除

- 从根节点开始，在每个节点处首先判断节点自身是否在灰色区域内，若是则标记删除 (deleted=true) ；
- 判断两个子节点的range是否与删除区域有交叉，若无则直接剪枝，

- 若有则重复上述过程；
- 直至搜索完所有的子节点，完成所有被删除节点的标记删除；
- 每次增量操作之后，对treesize, invalidnum, range, treedeleted进行更新
- 被删除的节点不会立即删除，而是令deleted=true，最后在re-balancing环节删掉



## 再平衡

$\alpha - blanced$  准则：

$$\begin{aligned} S(T.\text{leftchild}) &< \alpha_{bal}(S(T) - 1) \\ S(T.\text{rightchild}) &< \alpha_{bal}(S(T) - 1) \end{aligned} \quad (1)$$

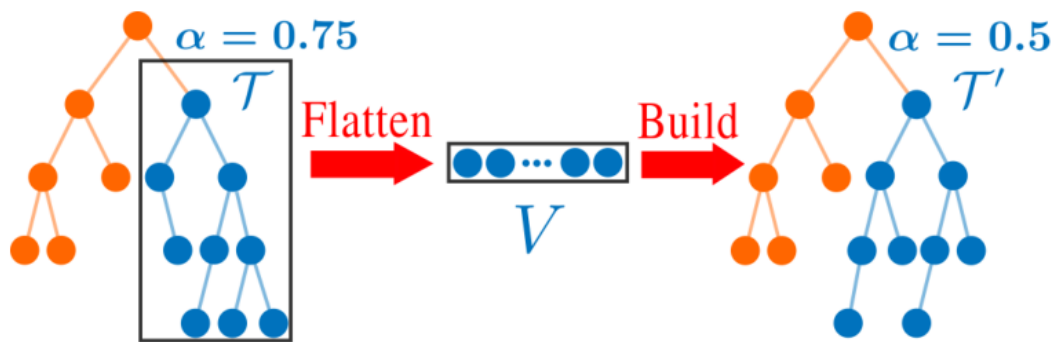
$\alpha_{bal} \in (0.5, 1)$ ， $S(T)$  是节点  $T$  的treesize。实际上是比较某个节点左右两个子树的规模，规模不能差的太多。当  $\alpha_{bal}$  越接近0.5，表明我们要求子树越趋近于“完美”平衡。

$\alpha - deleted$  准则：

$$I(T) < \alpha_{del} S(T) \quad (2)$$

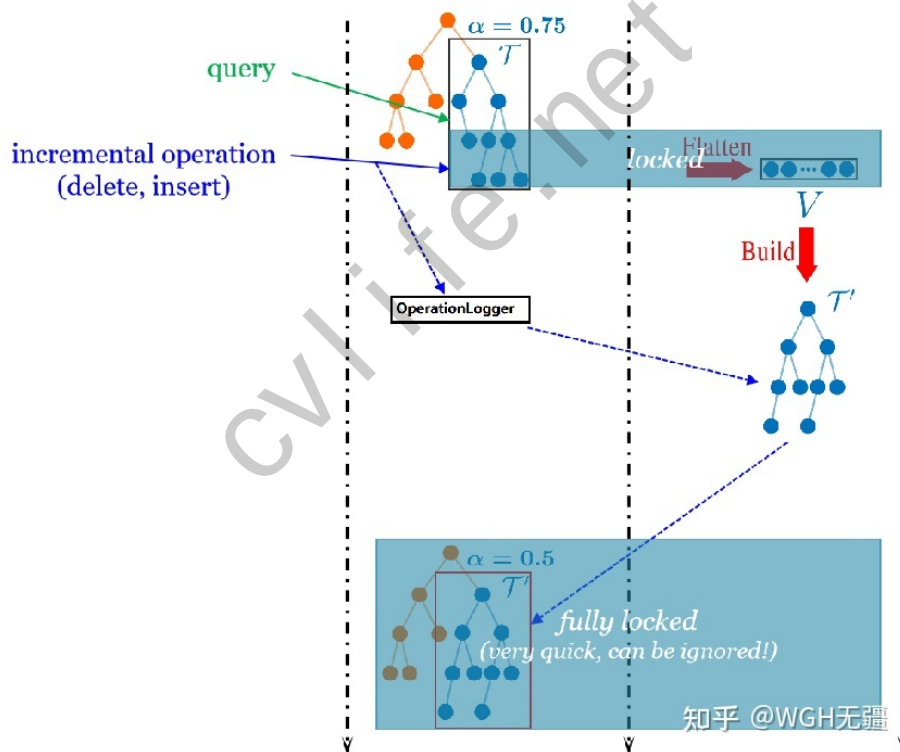
$I(T)$  表示子树上无效节点的数量。它描述的是我们最多能够容忍一个子树中有多大比例“被标记为删除、但是还没有事实删除的点”。违反准则的子树会被执行重构。

当重建的规模较小时，重建的时间开销可以忽略，直接在主线程中重建。具体为把子树打乱重排，去掉deleted=true的点，按照常规kdtree构建的方法重新构建即可。



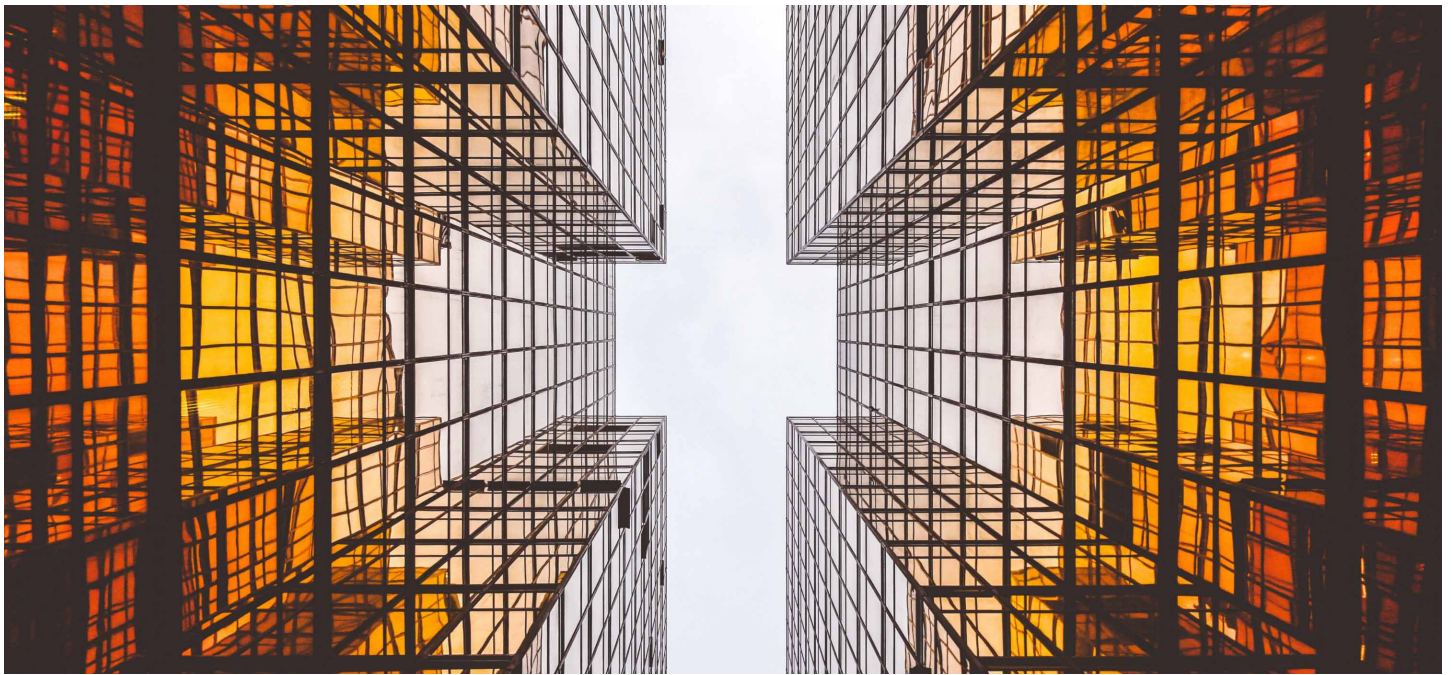
如果需要重建的(sub)tree规模很大,重建时间花销不可忽略;如果在主线程中执行重建,就会导致ikd-Tree一直被占用,外部的SLAM/LIO算法无法访问ikd-Tree执行查询或增删操作,导致算法阻塞。

针对这种情况,ikd-Tree采取了一种多线程策略。耗时的重建过程在额外的线程中执行,主线程仍旧对外提供查询和增删服务,其中涉及到“正在重建的(sub)tree”的增删操作会被额外缓存到一个叫作OperationLogger的容器中;待额外线程中完成了重建后,会从OperationLogger把增删操作“补作业”到新的(sub)tree上,这样就确保了新的(sub)tree没有漏掉任何过程;最后,把新子树替换到相应位置。

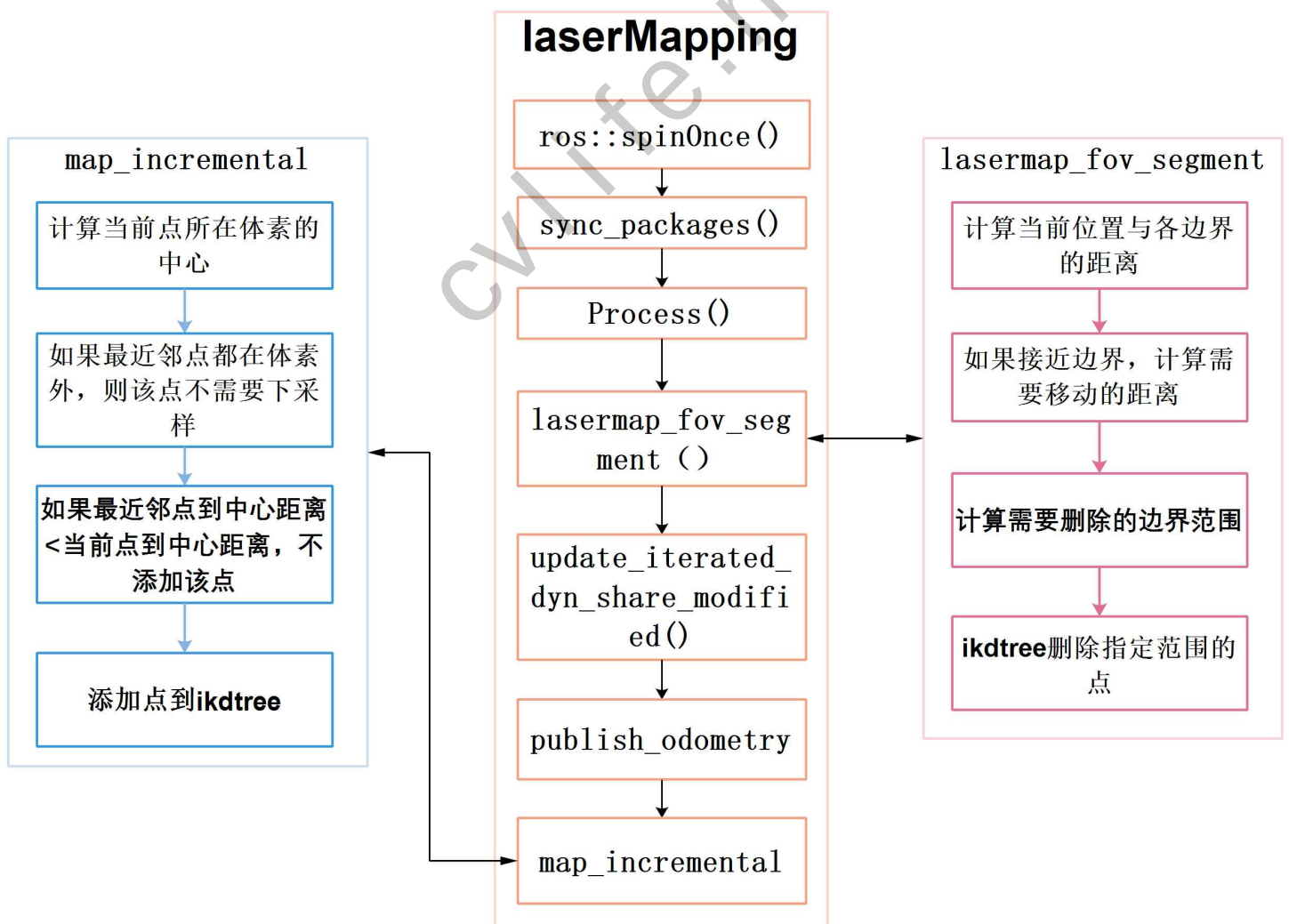


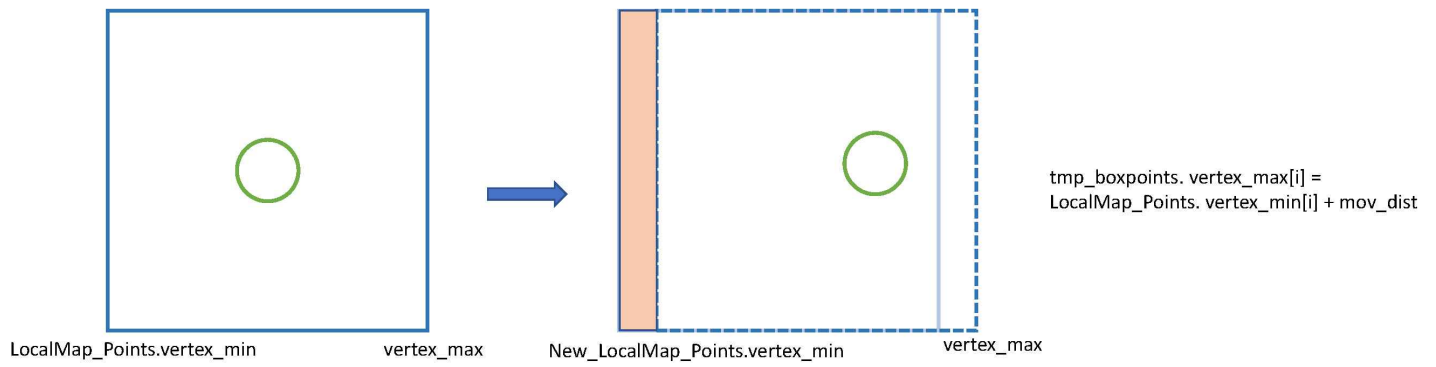
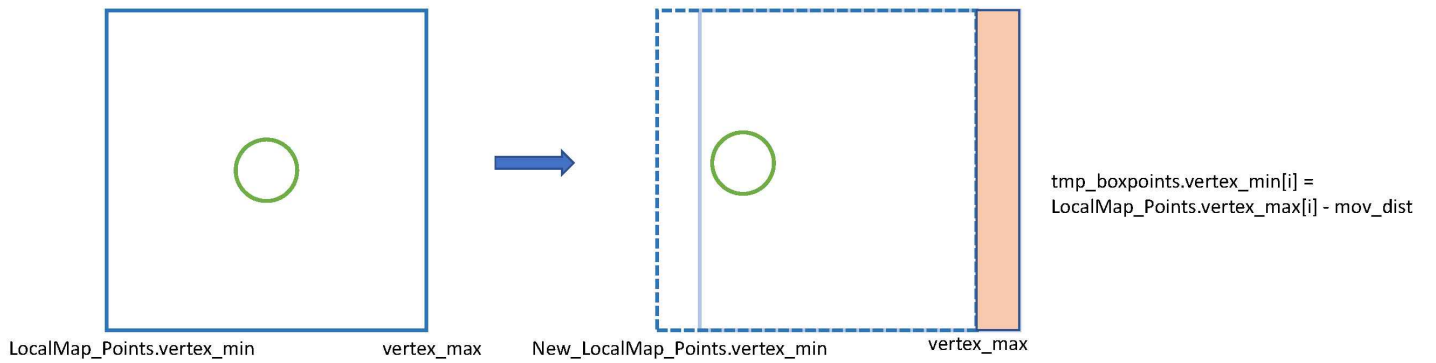
## 近邻点搜索

- 与kd-tree类似,但会利用range信息进行剪枝



## 16.代码讲解：建图与里程计发布





cvlife.net